

Entrada/Saída

- 7.1** Dispositivos externos
 - Teclado/monitor
 - Unidade de disco
- 7.2** Módulos de E/S
 - Função do módulo
 - Estrutura do módulo de E/S
- 7.3** E/S programada
 - Visão geral da E/S programada
 - Comandos de E/S
 - Instruções de E/S
- 7.4** E/S controlada por interrupção
 - Processamento de interrupção
 - Aspectos de projeto
 - Controlador de interrupção Intel 82C59A
 - A interface de periférico programável Intel 82C55A
- 7.5** Acesso direto à memória
 - Desvantagens da E/S programada e controlada por interrupção
 - Função do DMA
 - Controlador de DMA Intel 8237A
- 7.6** Canais e processadores de E/S
 - A evolução da função de E/S
 - Características dos canais de E/S
- 7.7** A interface externa: Firewire e InfiniBand
 - Tipos de interfaces
 - Configurações ponto a ponto e multiponto
 - Barramento serial FireWire
 - InfiniBand
- 7.8** Leitura recomendada e sites Web
 - Sites web recomendados

PRINCIPAIS PONTOS

- A arquitetura de E/S do sistema de computação é a sua interface com o mundo exterior. Essa arquitetura oferece um meio sistemático de controlar a interação com o mundo exterior e fornece ao sistema operacional as informações de que precisa para gerenciar a atividade de E/S de modo eficaz.
- Existem três técnicas principais de E/S: **E/S programada**, em que a E/S ocorre sob o controle direto e contínuo do programa solicitando a operação de E/S; **E/S controlada por interrupção**, em que um programa emite um comando de E/S e depois continua a executar, até que seja interrompido pelo hardware de E/S para sinalizar o final da operação de E/S; e **acesso direto à memória (DMA)**, em que um processador de E/S especializado assume o controle de uma operação de E/S para mover um grande bloco de dados.
- Dois exemplos importantes de interfaces de E/S são **FireWire** e **InfiniBand**.



Ferramenta de projeto de sistema de E/S

Além do processador e um conjunto de módulos de memória, o terceiro elemento chave de um sistema de computação é um conjunto de módulos de E/S. Cada módulo se conecta ao barramento do sistema ou **comutador** central e controla um ou mais dispositivos periféricos. Um módulo de E/S não é simplesmente um conjunto de conectores mecânicos que conectam um dispositivo fisicamente ao barramento do sistema. Em vez disso, o módulo de E/S contém uma lógica para realizar uma função de comunicação entre o periférico e o barramento.

O leitor poderá perguntar por que não se conectam os periféricos diretamente no barramento do sistema. Os motivos são os seguintes:

- Existe uma grande variedade de periféricos, com diversos métodos de operação. Seria impraticável incorporar a lógica necessária dentro do processador para controlar todos os tipos de dispositivos.
- A taxa de transferência de dados dos periféricos normalmente é muito mais lenta do que a da memória ou do processador. Assim, é impraticável usar o barramento de alta velocidade do sistema para se comunicar diretamente com um periférico.
- Por outro lado, a taxa de transferência de dados de alguns periféricos é maior do que a da memória ou do processador. Novamente, uma diferença levaria a ineficiências se não fosse controlada corretamente.
- Os periféricos normalmente utilizam formatos de dados e tamanhos de palavras diferentes do que é usado pelo computador ao qual estão conectados.

Assim, um módulo de E/S é necessário. Esse módulo tem duas funções principais (Figura 7.1):

- Interface com o processador e a memória por meio do barramento do sistema ou **comutador** central.
- Interface com um ou mais dispositivos periféricos por conexões de dados adequados.

Vamos começar este capítulo com uma breve discussão sobre os dispositivos externos, seguida por uma visão geral da estrutura e função de um módulo de E/S. Depois, veremos as diversas maneiras como a função de E/S pode ser realizada em cooperação com o processador e a memória: a interface de E/S interna. Finalmente, examinamos a interface de E/S externa, entre o módulo de E/S e o mundo exterior.

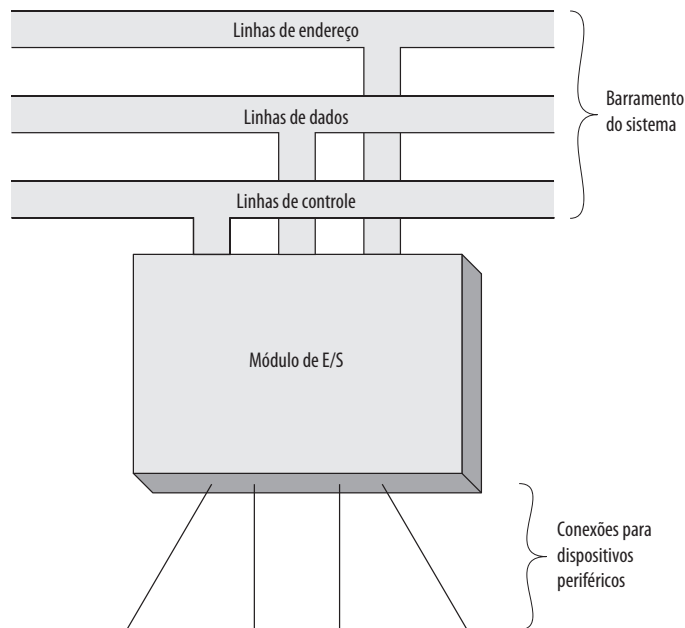


7.1 Dispositivos externos

As operações de E/S são realizadas por meio de uma grande variedade de dispositivos externos, que oferecem um meio de trocar dados entre o ambiente externo e o computador. Um dispositivo externo se conecta ao computador por uma conexão com um módulo de E/S (Figura 7.1). A conexão é usada para trocar sinais de controle, estado e dados entre os módulos de E/S e o dispositivo externo. Um dispositivo externo conectado a um módulo de E/S normalmente é chamado de **dispositivo periférico** ou, simplesmente, um **periférico**.

Podemos classificar os dispositivos externos em geral em três categorias:

Figura 7.1 Modelo genérico de um módulo de E/S



- **Legíveis ao ser humano:** adequados para a comunicação com usuários de computador.
- **Legíveis à máquina:** adequados para a comunicação com equipamentos.
- **Comunicação:** adequados para a comunicação com dispositivos remotos.

Alguns exemplos de dispositivos legíveis ao ser humano são monitores de vídeo e impressoras. Alguns exemplos de dispositivos legíveis à máquina são sistemas de disco magnético e fita, e sensores e atuadores, como aqueles usados em uma aplicação de robótica. Observe que estamos vendo os sistemas de disco e fita como dispositivos de E/S neste capítulo, enquanto, no Capítulo 6, eles são vistos como dispositivos de memória. Por um ponto de vista funcional, esses dispositivos fazem parte da hierarquia de memória, e seu uso é discutido apropriadamente no Capítulo 6. Por um ponto de vista estrutural, esses dispositivos são controlados por módulos de E/S e, por isso, devem ser considerados neste capítulo.

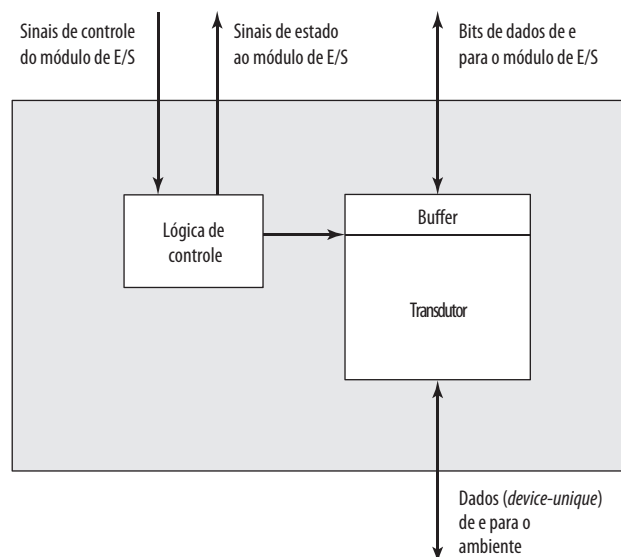
Dispositivos de comunicação permitem que um computador troque dados com um dispositivo remoto, que pode ser um dispositivo legível ao ser humano, como um terminal, um dispositivo legível à máquina, ou até mesmo outro computador.

Em termos muito gerais, a natureza de um dispositivo externo é indicada na Figura 7.2. A interface com o módulo de E/S ocorre na forma de sinais de controle, dados e estado. Os **sinais de controle** determinam a função que o dispositivo realizará como enviar dados ao módulo de E/S (INPUT ou READ), aceitar dados do módulo de E/S (OUTPUT ou WRITE), informar o estado ou realizar alguma função de controle particular ao dispositivo (por exemplo, posicionar uma cabeça de disco). Os **dados** estão na forma de um conjunto de bits a serem enviados ou recebidos do módulo de E/S. Os **sinais de estado** indicam o estado do dispositivo. Alguns exemplos são READY/NOT-READY, para indicar se o dispositivo está pronto para uma transferência de dados.

A **lógica de controle**, associada ao dispositivo, controla a operação do dispositivo em resposta à direção do módulo de E/S. O **transdutor** converte dados de elétrico para outras formas de energia durante a saída e de outras formas para elétrico durante a entrada. Normalmente, um buffer é associado ao transdutor para manter temporariamente os dados sendo transferidos entre o módulo de E/S e o ambiente externo; um tamanho de buffer de 8 a 16 bits é comum.

A interface entre o módulo de E/S e o dispositivo externo será examinada na Seção 7.7. A interface entre o dispositivo externo e o ambiente está fora do escopo deste livro, mas vamos mostrar alguns exemplos rapidamente.

Figura 7.2 Diagrama em blocos de um dispositivo externo





Teclado/monitor

O meio mais comum de interação entre computador/usuário é o conjunto de teclado/monitor. O usuário fornece entrada pelo teclado. Essa entrada é então transmitida ao computador e também pode ser exibida no monitor. Além disso, o monitor exibe dados fornecidos pelo computador.

A unidade de troca básica é o caractere. Associado a cada caractere existe um código, normalmente com 7 ou 8 bits. O código de texto mais utilizado é o *International Reference Alphabet* (IRA).¹ Cada caractere nesse código é representado por um código binário exclusivo com 7 bits; assim, 128 caracteres diferentes podem ser representados. Os caracteres são de dois tipos: imprimíveis e de controle. Os caracteres imprimíveis são os caracteres alfabéticos, numéricos e especiais, que podem ser impressos em papel ou exibidos em um monitor. Alguns dos caracteres de controle têm a ver com o controle da impressão ou exibição de caracteres; um exemplo é o *carriage return*. Outros caracteres de controle tratam dos procedimentos de comunicação. Veja mais detalhes no Apêndice F.

Para a entrada do teclado, quando o usuário pressiona uma tecla, isso gera um sinal eletrônico que é interpretado pelo transdutor no teclado e traduzido para o padrão de bits do código IRA correspondente. Esse padrão de bits é então transmitido ao módulo de E/S no computador, onde o texto pode ser armazenado no mesmo código IRA. Na saída, os caracteres do código IRA são transmitidos para um dispositivo externo do módulo de E/S. O transdutor no dispositivo interpreta esse código e envia os sinais eletrônicos exigidos ao dispositivo de saída, ou para exibir o caractere indicado ou realizar a função de controle solicitada.



Unidade de disco

Uma unidade de disco contém a eletrônica para trocar sinais de dados, controle e estado com um módulo de E/S mais a eletrônica para controlar os mecanismos de leitura/escrita de disco. Em um disco de cabeça fixa, o transdutor é capaz de converter os padrões magnéticos na superfície do disco móvel em bits no buffer do dispositivo (Figura 7.2). Um disco com cabeça móvel também precisa ser capaz de fazer o braço do disco se mover radialmente para dentro e fora pela superfície do disco.



7.2 Módulos de E/S



Função do módulo

As principais funções ou requisitos para um módulo de E/S encontram-se nas seguintes categorias:

- Controle e temporização.
- Comunicação com o processador.
- Comunicação com o dispositivo.
- Armazenamento temporário (*buffering*) de dados.
- Detecção de erro

Durante qualquer período, o processador pode se comunicar com um ou mais dispositivos externos em padrões imprevisíveis, dependendo da necessidade de E/S do programa. Os recursos internos, como a memória principal e o barramento do sistema, precisam ser compartilhados entre uma série de atividades, incluindo E/S de dados. Assim, a função de E/S inclui um requisito de **controle e temporização**, para coordenar o fluxo de tráfego entre os recursos internos e dispositivos externos. Por exemplo, o controle da transferência de dados de um dispositivo externo ao processador poderia envolver a seguinte sequência de etapas:

1. O processador interroga o módulo de E/S para verificar o estado do dispositivo conectado.
2. O módulo de E/S retorna o estado do dispositivo.
3. Se o dispositivo estiver operacional e pronto para transmitir, o processador solicita a transferência de dados por meio de um comando ao módulo de E/S.

¹ IRA é definido na ITU-T Recommendation T.50, e era conhecido originalmente como *International Alphabet Number 5* (IA5). A versão do IRA para os EUA é conhecida como *American Standard Code for Information Interchange* (ASCII).

4. O módulo de E/S obtém uma unidade de dados (por exemplo, 8 ou 16 bits) do dispositivo externo.
5. Os dados são transferidos do módulo de E/S ao processador.

Se o sistema emprega um barramento, então cada uma das interações entre o processador e o módulo de E/S envolve uma ou mais arbitragens de barramento.

Esse cenário simplificado também ilustra que o módulo de E/S precisa se comunicar com o processador e com o dispositivo externo. A **comunicação do processador** envolve o seguinte:

- **Decodificação de comando:** o módulo de E/S aceita comandos do processador, normalmente enviados como sinais no barramento de controle. Por exemplo, um módulo de E/S para uma unidade de disco precisa aceitar os seguintes comandos: READ SECTOR, WRITE SECTOR, SEEK número de trilha e SCAN ID de registro. Cada um dos dois últimos comandos inclui um parâmetro que é enviado no barramento de dados.
- **Dados:** os dados são trocados entre o processador e o módulo de E/S pelo barramento de dados.
- **Informação de estado:** como os periféricos são muito lentos, é importante conhecer o estado do módulo de E/S. Por exemplo, se um módulo de E/S tiver que enviar dados ao processador (leitura), ele pode não ser capaz de fazer isso porque ainda está trabalhando no comando de E/S anterior. Esse fato pode ser relatado com um sinal de estado, sendo os mais comuns BUSY e READY. Também pode haver sinais para relatar diversas condições de erro.
- **Reconhecimento de endereço:** assim como cada palavra de memória tem um endereço, cada dispositivo de E/S também tem. Desse modo, um módulo de E/S precisa reconhecer um endereço exclusivo para cada periférico que controla.

Por outro lado, o módulo de E/S também deve ser capaz de realizar **comunicação com o dispositivo**. Essa comunicação envolve comandos, informação de estado e dados (Figura 7.2).

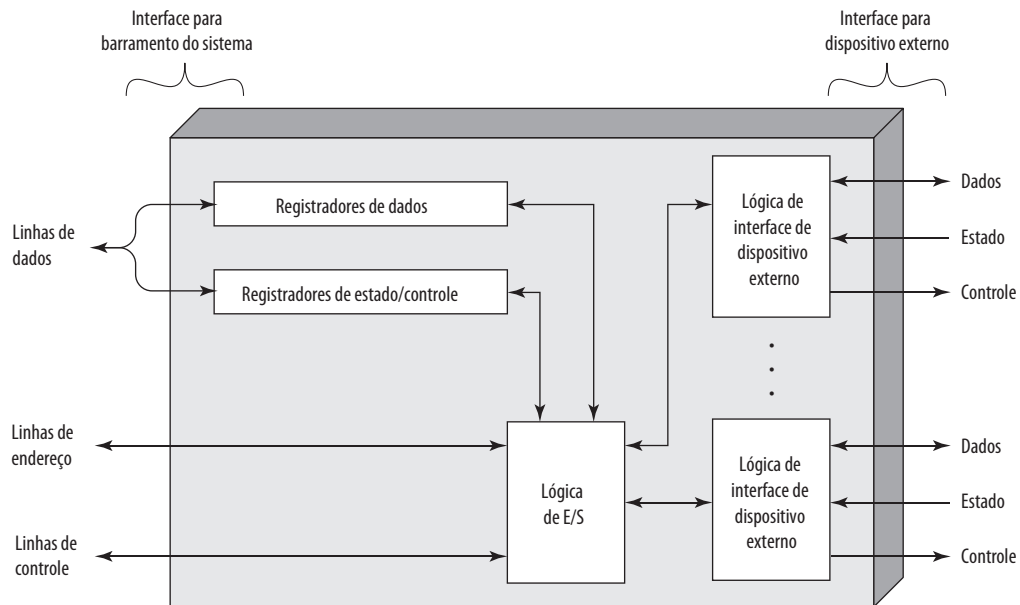
Uma tarefa essencial de um módulo de E/S é o **buffering de dados**. A necessidade dessa função é aparente pela Figura 2.11. Enquanto a taxa de transferência para entrada e saída na memória principal ou no processador é muito alta, as taxas da maioria dos dispositivos periféricos compreendem uma grande faixa. Os dados vindos da memória principal são enviados para um módulo de E/S em uma maneira rápida. Os dados são mantidos em um buffer no módulo de E/S e depois enviados ao dispositivo periférico em sua taxa de dados. Na direção oposta, os dados são mantidos em buffer para não reter a memória com uma operação de transferência lenta. Assim, o módulo de E/S precisa ser capaz de operar nas velocidades do dispositivo e da memória. De modo semelhante, se o dispositivo de E/S opera em uma taxa mais alta que a taxa de acesso à memória, então o módulo de E/S realiza a operação de **buffering** necessária.

Finalmente, um módulo de E/S normalmente é responsável pela **deteção de erro** e, subsequentemente, por relatar erros ao processador. Uma classe de erros inclui defeitos mecânicos e elétricos relatados pelo dispositivo (por exemplo, papel emperrado, trilha de disco com defeito). Outra classe consiste em mudanças não intencionais no padrão de bits quando são transmitidos do dispositivo ao módulo de E/S. Alguma forma de código de detecção de erro normalmente é usada para detectar erros de transmissão. Um exemplo simples é o uso de um bit de paridade em cada caractere de dados. Por exemplo, o código de caracteres IRA ocupa 7 bits de um byte. O oitavo bit é definido de modo que o número total de 1s no byte seja par (paridade par) ou ímpar (paridade ímpar). Quando um byte é recebido, o módulo de E/S verifica a paridade para determinar se ocorreu um erro.



Estrutura do módulo de E/S

Os módulos de E/S variam bastante em complexidade e o número de dispositivos externos controlados por eles. Aqui, tentaremos apenas dar uma descrição. (Um dispositivo específico, o Intel 82C55A, é descrito na Seção 7.4.) A Figura 7.3 oferece um diagrama de blocos geral de um módulo de E/S. O módulo se conecta ao restante do computador por meio de um conjunto de linhas de sinal (por exemplo, linhas de barramento do sistema). Os dados transferidos de e para o módulo são mantidos em um buffer, em um ou mais registradores de dados. Também pode haver um ou mais registradores de estado que oferecem informações do estado atual. Um registrador de estado também pode funcionar como um registrador de controle, para aceitar informações de controle detalhadas do processador. A lógica dentro do módulo interage com o processador por meio de um conjunto de linhas de controle. O processador usa as linhas de controle para emitir comandos ao módulo de E/S. Algumas das linhas de controle podem ser usadas pelo módulo de E/S (por exemplo, para sinais de arbitração

Figura 7.3 Diagrama de blocos de um módulo de E/S

e estado). O módulo também precisa ser capaz de reconhecer e gerar endereços associados aos dispositivos que ele controla. Cada módulo de E/S tem um endereço exclusivo ou, se controlar mais de um dispositivo externo, um conjunto exclusivo de endereços. Finalmente, o módulo de E/S contém lógica específica à interface com cada dispositivo que ele controla.

Um módulo de E/S funciona para permitir que o processador veja uma grande variedade de dispositivos de uma maneira simples. Existe um espectro de capacidades que podem ser oferecidas. O módulo de E/S pode ocultar os detalhes de temporização, formatos e eletromecânica de um dispositivo externo, de modo que o processador pode funcionar em termos de comandos simples de leitura e escrita, e possivelmente comandos para abrir e fechar arquivo. Em sua forma mais simples, o módulo de E/S ainda pode ter grande parte do trabalho de controlar um dispositivo (por exemplo, rebobinar uma fita) visível ao processador.

Um módulo de E/S, que assume a maior parte do processamento, apresentando uma interface de alto nível ao processador, normalmente é conhecido como *canal de E/S* ou *processador de E/S*. Um módulo de E/S que é muito primitivo e requer controle normalmente é conhecido como *controlador de E/S* ou *controlador de dispositivo*. Os controladores de E/S normalmente são vistos nos microcomputadores, enquanto os canais de E/S são usados em mainframes.

A seguir, usaremos o termo genérico *módulo de E/S* quando não houver confusão, e usaremos termos mais específicos onde for necessário.



7.3 E/S programada

Três técnicas são possíveis para operações de E/S. Com a *E/S programada*, os dados são trocados entre o processador e o módulo de E/S. O processador executa um programa que lhe oferece controle direto da operação de E/S, incluindo percepção do estado de dispositivo, envio de um comando de leitura ou escrita e transferência dos dados. Quando o processador emite um comando ao módulo de E/S, ele precisa esperar até que a operação de E/S termine. Se o processador for mais rápido que o módulo de E/S, isso desperdiça o tempo do processador. Com a *E/S controlada por interrupção*, o processador emite um comando de E/S, continua a executar outras instruções e é interrompido pelo módulo de E/S quando o último tiver completado seu trabalho. Com a E/S pro-

gramada e por interrupção, o processador é responsável por obter dados da memória principal para saída, e por armazenar dados na memória principal para entrada. A alternativa é conhecida como **acessos direto à memória** (DMA). Nesse modo, o módulo de E/S e a memória principal trocam dados diretamente, sem envolvimento do processador.

A Tabela 7.1 indica a relação entre essas três técnicas. Nesta seção, exploramos a E/S programada. A E/S por interrupção e DMA são exploradas nas duas seções seguintes, respectivamente.



Visão geral da E/S programada

Quando o processador está executando um programa e encontra uma instrução relacionada a E/S, ele executa essa instrução emitindo um comando ao módulo de E/S apropriado. Com a E/S programada, o módulo de E/S realizará a ação exigida e depois definirá os bits apropriados no registrador de estado de E/S (Figura 7.3). O módulo de E/S não toma outra ação para alertar o processador. Em particular, ele não interrompe o processador. Assim, é responsabilidade do processador verificar periodicamente o estado do módulo de E/S até descobrir se a operação terminou.

Para explicar a técnica de E/S programada, primeiramente a veremos do ponto de vista dos comandos de E/S emitidos pelo processador ao módulo de E/S, e depois do ponto de vista das instruções de E/S executadas pelo processador.



Comandos de E/S

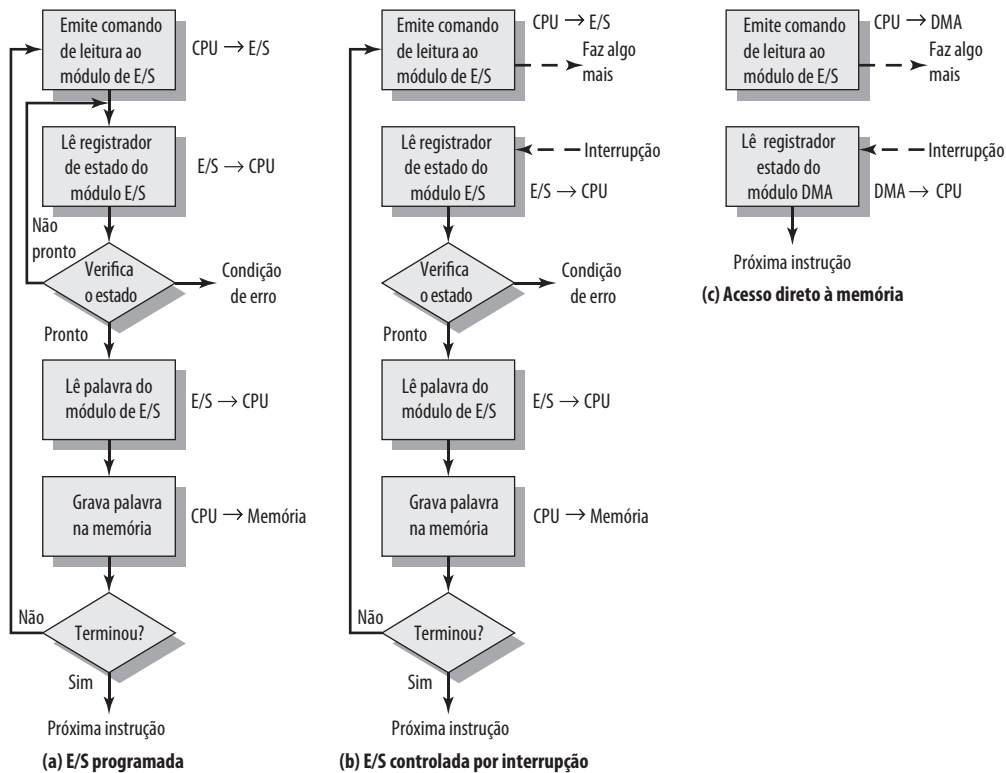
Para executar uma instrução relacionada a E/S, o processador emite um endereço, especificando o módulo de E/S e dispositivo externo em particular, e um comando de E/S. Existem quatro tipos de comandos de E/S que um módulo de E/S pode receber quando é endereçado por um processador:

- **Controle:** usado para ativar um periférico e dizer-lhe o que fazer. Por exemplo, uma unidade de fita magnética pode ser instruída a rebobinar ou mover um registrador para frente. Esses comandos são ajustados ao tipo específico de dispositivo periférico.
- **Teste:** usado para testar diversas condições de estado associadas a um módulo de E/S e seus periféricos. O processador desejará saber se o periférico de interesse está ligado e disponível para uso. Ele também deseja saber se a operação de E/S mais recente terminou e se houve algum erro.
- **Leitura:** faz com que o módulo de E/S obtenha um item de dados do periférico e o coloque em um buffer interno (representado como um registrador de dados na Figura 7.3). O processador pode obter o item de dados solicitando que o módulo de E/S o coloque no barramento de dados.
- **Escrita:** faz com que o módulo de E/S apanhe um item de dado (byte ou palavra) do barramento de dados e depois transmita esse item de dado ao periférico.

A Figura 7.4a dá um exemplo do uso da E/S programada para ler um bloco de dados de um dispositivo periférico (por exemplo, um registro da fita) para a memória. Os dados são lidos em uma palavra (por exemplo, 16 bits) de cada vez. Para cada palavra lida, o processador precisa permanecer em um ciclo de verificação de estado até que determine que a palavra está disponível no registrador de dados do módulo de E/S. Esse fluxograma destaca a principal desvantagem dessa técnica: é um processo demorado, que mantém o processador ocupado desnecessariamente.

Tabela 7.1 Técnicas de E/S

	Sem interrupções	Uso de interrupções
Transferência de E/S para memória via processador	E/S programada	E/S controlada por interrupção
Transferência direta de E/S para memória		Acesso direto à memória (DMA)

Figura 7.4 Três técnicas para entrada de um bloco de dados

Instruções de E/S

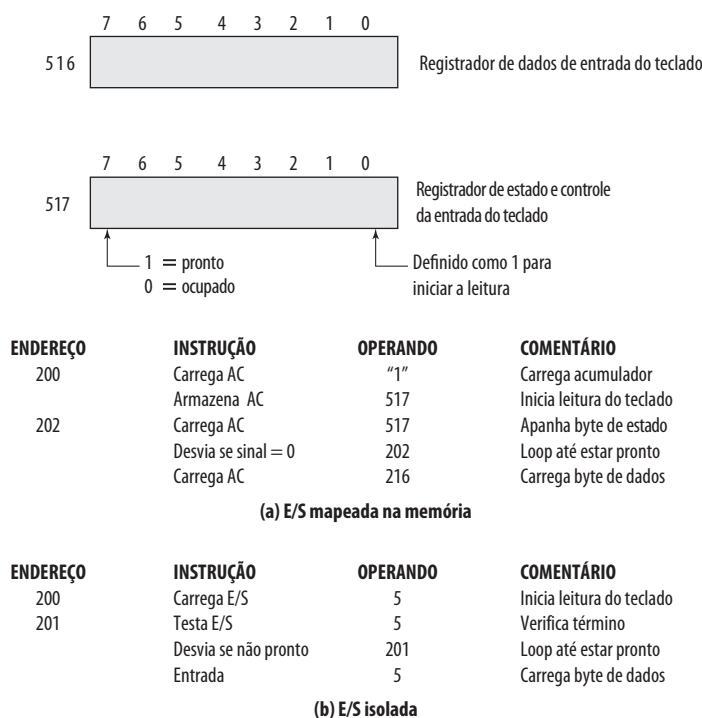
Com a E/S programada, existe uma correspondência próxima entre as instruções relacionadas à E/S que o processador busca na memória e os comandos de E/S que o processador emite a um módulo de E/S, para executar as instruções. Ou seja, as instruções são facilmente mapeadas em comandos de E/S, e normalmente existe uma simples relação um para um. A forma da instrução depende do modo como os dispositivos externos são endereçados.

Normalmente, haverá muitos dispositivos de E/S conectados por meio dos módulos de E/S ao sistema. Cada dispositivo recebe um identificador ou endereço exclusivo. Quando o processador emite um comando de E/S, o comando contém o endereço do dispositivo desejado. Assim, cada módulo de E/S precisa interpretar as linhas de endereço para determinar se o comando é para ele mesmo.

Quando o processador, a memória principal e a E/S compartilham um barramento comum, dois modos de endereçamento são possíveis: E/S mapeada na memória e E/S independente. Com a **E/S mapeada na memória**, existe um único espaço de endereço para locais de memória e dispositivos de E/S. O processador trata os registradores de estado e dados dos módulos de E/S como locais de memória, e usa as mesmas instruções de máquina para acessar a memória e os dispositivos de E/S. Assim, por exemplo, com 10 linhas de endereço, um total combinado de $2^{10} = 1024$ locais de memória e endereços de E/S podem ser aceitos, em qualquer combinação.

Com a E/S mapeada na memória, uma única linha de leitura e uma única linha de escrita são necessárias no barramento. Como alternativa, o barramento pode ter linhas de leitura e escrita de memória além das linhas de comando de entrada e saída. Agora, a linha de comando especifica se o endereço se refere a um local de memória ou a um dispositivo de E/S. A faixa completa de endereços pode estar disponível para ambos. Novamente, com 10 linhas de endereço, o sistema agora pode aceitar 1024 locais de memória e 1024 endereços de E/S. Como o espaço de endereço para E/S é independente do espaço da memória, isso é chamado de **E/S independente**.

A Figura 7.5 compara essas duas técnicas de E/S programada. A Figura 7.5a mostra como a interface para um dispositivo de entrada simples, como um teclado de, poderia aparecer a um programador usando a E/S mapeada

Figura 7.5 E/S mapeada na memória e isolada

na memória. Considere um endereço de 10 bits, com uma memória de 512 bits (loais 0-511) e até 512 endereços de E/S (loais 512-1023). Dois endereços são dedicados à entrada do teclado de um terminal em particular. O endereço 516 refere-se ao registrador de dados e o endereço 517 refere-se ao registrador de estado, que também funciona como um registrador de controle para receber comandos do processador. O programa mostrado lerá 1 byte de dados do teclado para um registrador acumulador no processador. Observe que o processador entra em um loop até que o byte de dados esteja disponível.

Com a E/S independente (Figura 7.5b), as portas de E/S são acessíveis apenas por comandos de E/S especiais, que ativam as linhas de comando de E/S no barramento.

Para a maioria dos tipos de processadores, existe um conjunto relativamente grande de diferentes instruções para referenciar a memória. Se a E/S independente for usada, haverá apenas algumas instruções de E/S. Assim, uma vantagem da E/S mapeada na memória é que esse grande repertório de instruções pode ser usado, permitindo uma programação mais eficiente. Uma desvantagem é que um espaço de endereços de memória valioso é utilizado. Tanto a E/S mapeada na memória quanto a E/S independente são comumente utilizadas.



7.4 E/S controlada por interrupção

O problema com a E/S programada é que o processador tem que esperar muito tempo para que o módulo de E/S de interesse esteja pronto para recepção ou transmissão de dados. O processador, enquanto espera, precisa interrogar repetidamente o estado do módulo de E/S. Como resultado, o nível de desempenho do sistema inteiro é bastante degradado.

Uma alternativa é que o processador emita um comando de E/S para um módulo e depois continue realizando algum outro trabalho útil. O módulo de E/S, então, interromperá o processador para solicitar atendimento quando estiver pronto para trocar dados com o processador. O processador, então, executa a transferência de dados, como antes, e depois retoma seu processamento anterior.

Vamos considerar como isso funciona, primeiro do ponto de vista do módulo de E/S. Para a entrada, o módulo de E/S recebe um comando READ do processador. O módulo de E/S, então, prossegue para ler dados de um periférico associado. Quando os dados estão no registrador de dados do módulo, o módulo envia um sinal de interrupção ao processador por uma linha de controle. O módulo, então, espera até que seus dados sejam solicitados pelo processador. Quando a solicitação termina, o módulo coloca seus dados no barramento de dados e então está pronto para outra operação de E/S.

Do ponto de vista do processador, a ação para entrada é a seguinte. O processador emite um comando READ. Depois, ele prossegue com outras tarefas (por exemplo, o processador pode estar trabalhando em vários programas diferentes ao mesmo tempo). Ao final de cada ciclo de instrução, o processador verifica se há interrupções (Figura 3.9). Quando ocorre uma interrupção do módulo de E/S, o processador salva o contexto (por exemplo, o contador de programa e os registradores do processador) do programa atual e processa a interrupção. Nesse caso, o processador lê a palavra de dados do módulo de E/S e a armazena na memória. Depois, ele restaura o contexto do programa em que estava trabalhando (ou de algum outro programa) e retoma a execução.

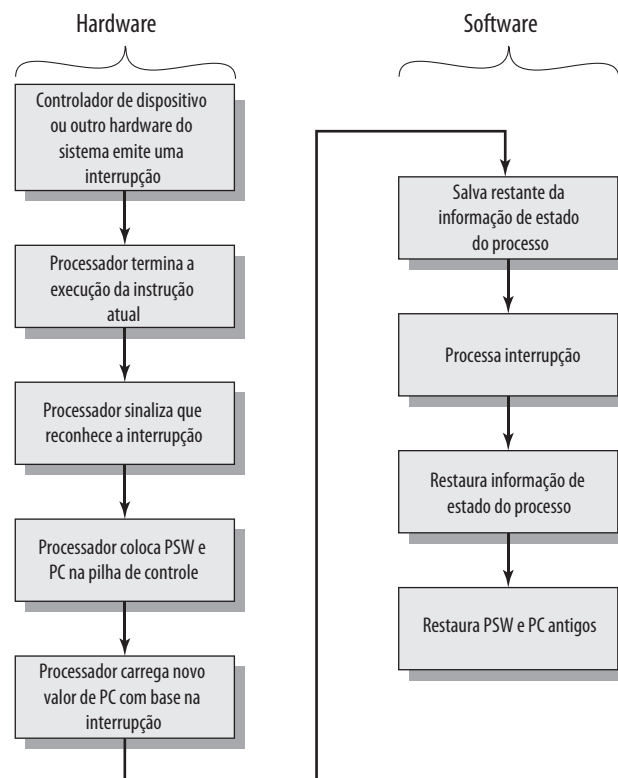
A Figura 7.4b mostra o uso da E/S por interrupção para leitura de um bloco de dados. Compare isso com a Figura 7.4a. A E/S por interrupção é mais eficiente do que a E/S programada, pois elimina a espera desnecessária. Porém, a E/S por interrupção ainda consome muito tempo do processador, pois cada palavra de dados que vem da memória para o módulo de E/S ou do módulo de E/S para a memória precisa passar pelo processador.



Processamento de interrupção

Vamos examinar com mais detalhes o papel do processador na E/S controlada por interrupção. O surgimento de uma interrupção dispara uma série de eventos, tanto no hardware do processador quanto no software. A Figura 7.6 mostra uma sequência típica. Quando o dispositivo de E/S completa uma operação de E/S, ocorre a seguinte sequência de eventos de hardware:

Figura 7.6 Processamento de interrupção simples



1. O dispositivo emite um sinal de interrupção ao processador.
2. O processador termina a execução da instrução atual antes de responder à interrupção, conforme indicado na Figura 3.9.
3. O processador testa uma interrupção, determina que existe interrupção e envia um sinal de confirmação ao dispositivo que a emitiu. A confirmação permite que o dispositivo remova seu sinal de interrupção.
4. O processador agora precisa se preparar para transferir o controle à rotina de interrupção. Para começar, ele precisa salvar as informações necessárias para retornar ao programa atual no ponto da interrupção. A informação mínima exigida é (a) o estado do processador, que está contido em um registrador chamado palavra de estado do programa (PSW — *program status word*), e (b) o local da próxima instrução a ser executada, que está contida no contador de programa. Estas podem ser colocadas na pilha de controle do sistema.²
5. O processador agora carrega o contador de programa com o local endereço inicial da rotina de tratamento de interrupção que responderá a essa interrupção. Dependendo da arquitetura de comunicação e do projeto do sistema operacional, pode haver uma única rotina, uma rotina para cada tipo de interrupção ou uma rotina para cada dispositivo e cada tipo de interrupção. Se houver mais de uma rotina de tratamento de interrupção, o processador precisa determinar qual irá chamar. Essa informação pode ter sido incluída no sinal de interrupção original, ou o processador pode ter que emitir uma solicitação ao dispositivo que emitiu a interrupção, para obter uma resposta que contém a informação necessária.

Quando o contador de programa tiver sido carregado, o processador segue para o próximo ciclo de instrução, que começa com uma busca de instrução. Como a busca de instrução é determinada pelo conteúdo do contador de programa, o resultado é que o controle é transferido para o programa manipulador de interrupção. A execução desse programa resulta nas seguintes operações:

6. Nesse ponto, o contador de programa e PSW relacionados ao programa interrompido foram salvos na pilha do sistema. Porém, existe outra informação que é considerada parte do “estado” do programa em execução. Em particular, os conteúdos dos registradores do processador precisam ser salvos, pois esses registradores podem ser usados pela rotina de tratamento de interrupção. Assim, todos os valores, mais qualquer outra informação de estado, precisam ser salvos. Normalmente, a rotina de tratamento de interrupção começará salvando o conteúdo dos registradores na pilha. A Figura 7.7a mostra um exemplo simples. Nesse caso, um programa do usuário é interrompido após a instrução no local N . O conteúdo de todos os registradores mais o endereço da próxima instrução ($N + 1$) são colocados na pilha. O ponteiro de pilha é atualizado para apontar para o novo topo da pilha, e o contador de programa é atualizado para apontar para o início da rotina de tratamento de interrupção.
7. A rotina de tratamento de interrupção em seguida processa a interrupção. Isso inclui uma verificação da informação de estado relacionada à operação de E/S ou outro evento que causou uma interrupção. Ele também pode envolver o envio de comandos ou confirmações adicionais ao dispositivo de E/S.
8. Quando o processamento da interrupção termina, os valores dos registradores salvos são recuperados da pilha e restaurados aos registradores (por exemplo, veja Figura 7.7b).
9. O ato final é restaurar os valores do PSW e contador de programa da pilha. Como resultado, a próxima instrução a ser executada será do programa previamente interrompido.

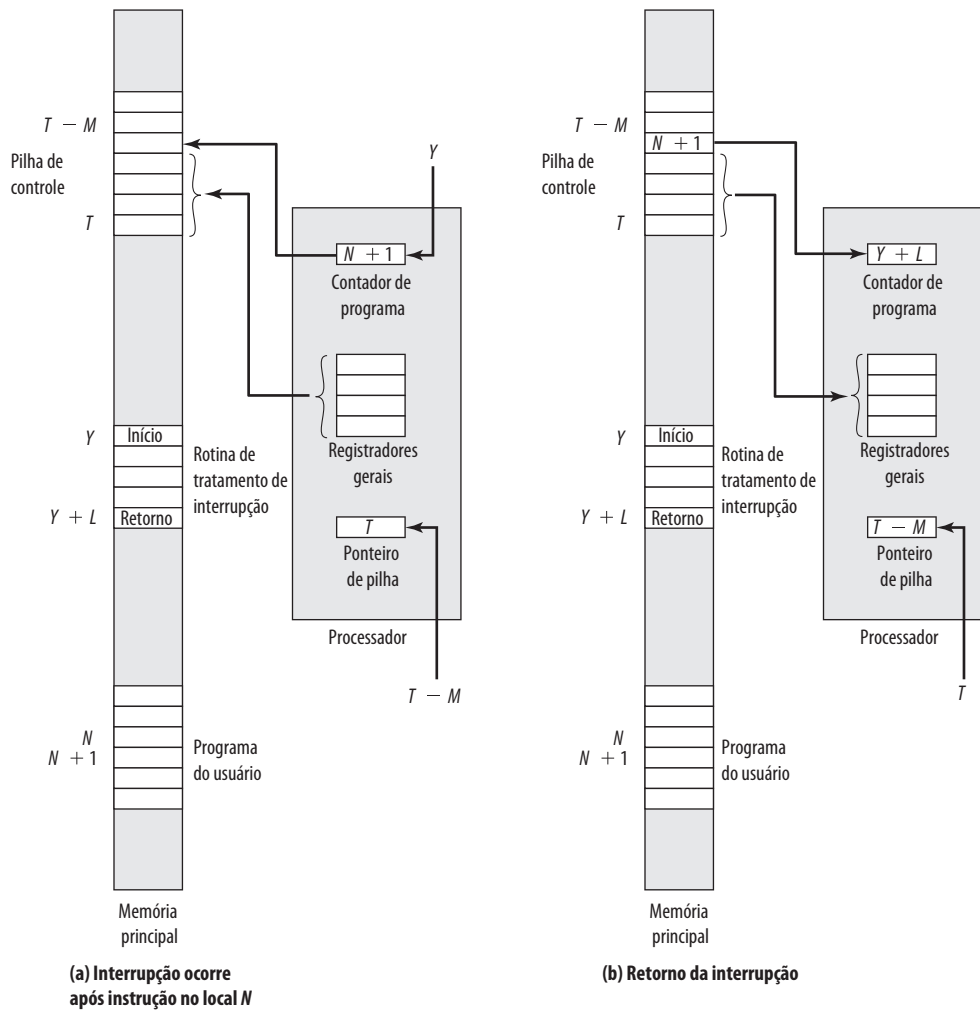
Observe que é importante salvar toda a informação de estado do programa interrompido para a retomada posterior, porque a interrupção não é uma rotina chamada pelo programa. Em vez disso, ela pode ocorrer a qualquer momento e, portanto, em qualquer ponto na execução de um programa do usuário. Sua ocorrência é imprevisível. Na verdade, conforme veremos no próximo capítulo, os dois programas podem não ter algo em comum e podem pertencer a dois usuários diferentes.



Aspectos de projeto

Dois aspectos de projeto surgem na implementação da E/S por interrupção. Primeiro, como quase sempre haverá vários módulos de E/S, como o processador determina qual dispositivo emitiu a interrupção? E segundo, se houver várias interrupções, como o processador decide qual deverá processar?

² Veja no Apêndice 10A uma discussão sobre a operação da pilha.

Figura 7.7 Mudanças na memória e registradores para uma interrupção

Vamos considerar primeiro a identificação do dispositivo. Quatro categorias gerais de técnicas são comumente utilizadas:

- Múltiplas linhas de interrupção.
- Verificação por software (polling).
- Daisy (verificação por hardware, vetorado).
- Arbitração de barramento (vetorado).

A técnica mais simples para o problema é oferecer **múltiplas linhas de interrupção** entre o processador e os módulos de E/S. Porém, é impraticável dedicar mais do que algumas poucas linhas de barramento ou pinos de processador às linhas de interrupção. Consequentemente, mesmo que várias linhas sejam usadas, provavelmente cada uma terá múltiplos módulos de E/S conectados a ela. Assim, uma das outras três técnicas precisa ser usada em cada linha.

Uma alternativa é a **verificação por software**. Quando o processador detecta uma interrupção, ele desvia para uma rotina de tratamento de interrupção cuja tarefa é verificar cada módulo de E/S para determinar qual módulo causou a interrupção. A verificação poderia ser por uma linha de comando separada (por exemplo, TESTI/O). Nesse caso, o processador atura TESTI/O e coloca o endereço de um módulo de E/S em particular nas linhas de endereço. O módulo de E/S responde positivamente solicitou a interrupção. Como alternativa, cada módulo de E/S poderia

conter um registrador de estado endereçável. O processador, então, lê o registrador de estado de cada módulo de E/S para identificar o módulo que gerou a interrupção. Quando o módulo for identificado, o processador inicia a execução da rotina de tratamento de interrupção para este dispositivo.

A desvantagem da verificação por software é que ele é demorado. Uma técnica mais eficiente é usar uma **Daisy chain**, que oferece uma verificação por hardware. Um exemplo de uma configuração Daisy chain aparece na Figura 3.26. Para interrupções, todos os módulos de E/S compartilham uma linha de requisição de interrupção comum. A linha de reconhecimento de interrupção é estruturada em forma de uma cadeia circular (em forma de margarida – em inglês, *daisy*) através dos módulos. Quando o processador reconhece uma interrupção, ele envia uma confirmação de interrupção. Esse sinal se propaga por uma série de módulos de E/S até que o módulo requisitante o receba. O módulo requisitante normalmente responde colocando uma palavra nas linhas de dados. Essa palavra é conhecida como **vetor de interrupção**, e é o endereço do módulo de E/S ou algum outro identificador exclusivo. De qualquer forma, o processador usa o vetor como um ponteiro para a rotina de serviço de dispositivo apropriada. Isso evita a necessidade de executar uma rotina de tratamento de interrupção geral primeiro. Essa técnica é chamada de **interrupção vetorada**.

Existe outra técnica que utiliza interrupções vetorizadas, que é a **arbitração de barramento**. Com a arbitração de barramento, um módulo de E/S precisa primeiro ganhar o controle do barramento antes de poder ativar a requisição de interrupção. Assim, somente um módulo pode ativar a linha de cada vez. Quando o processador detecta a interrupção, ele responde na linha de reconhecimento de interrupção. O módulo requisitante, então, coloca seu vetor nas linhas de dados.

As técnicas que mencionamos servem para identificar o módulo de E/S requisitante. Elas também oferecem um modo de atribuir prioridades quando mais de um dispositivo está requisitando serviço de interrupção. Com múltiplas linhas, o processador apenas apanha a linha de interrupção com a prioridade mais alta. Com o *polling* de software, a ordem em que os módulos são verificados determina suas prioridades. De modo semelhante, a ordem dos módulos em uma Daisy chain determina suas prioridades. Finalmente, a arbitração de barramento pode empregar um esquema de prioridade, conforme discutimos na Seção 3.4.

Agora, vamos examinar dois exemplos de estruturas de interrupção.



Controlador de interrupção Intel 82C59A

O Intel 80386 oferece uma única *Interrupt Request* (INTR) e uma única linha *Interrupt Acknowledge* (INTA). Para permitir que o 80386 trate de diversos dispositivos e estruturas de prioridade, ele normalmente é configurado com um árbitro de interrupção externo, o 82C59A. Os dispositivos externos são conectados ao 82C59A, que, por sua vez, se conecta ao 80386.

A Figura 7.8 mostra o uso do 82C59A para conectar múltiplos módulos de E/S para o 80386. Um único 82C59A pode tratar de até oito módulos. Se for preciso controlar mais de oito módulos, um arranjo em cascata pode ser usado, para tratar de até 64 módulos.

A única responsabilidade do 82C59A é o gerenciador de interrupções. Ele aceita requisições de interrupção dos módulos conectados, determina qual interrupção tem a maior prioridade e depois sinaliza o processador levantando a linha INTR. O processador confirma por meio da linha INTA. Isso pede ao 82C59A para colocar a informação de vetor apropriada no barramento de dados. O processador pode, então, prosseguir para processar a interrupção e se comunicar diretamente com o módulo de E/S para ler ou escrever dados.

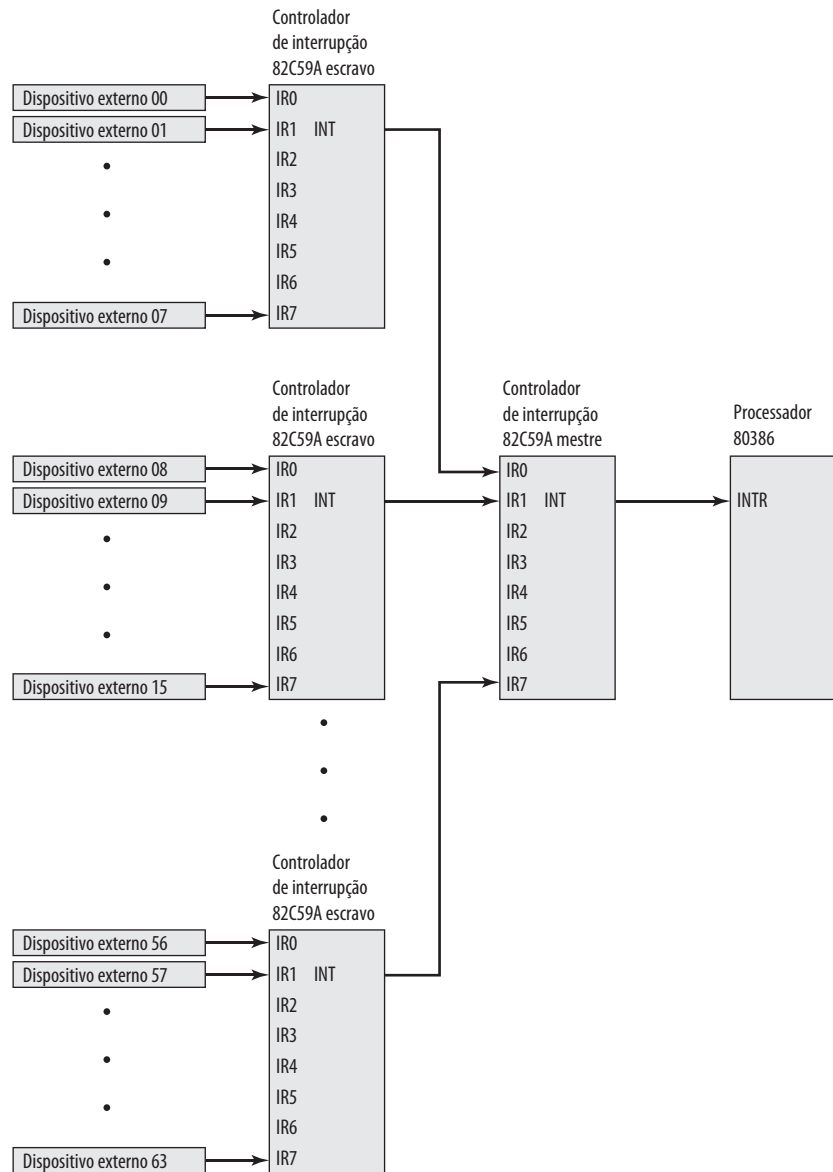
O 82C59A é programável. O 80386 determina o esquema de prioridade a ser usado definindo uma palavra de controle no 82C59A. Os seguintes modos de interrupção são possíveis:

- **Totalmente aninhado:** as requisições de interrupção são ordenadas na prioridade de 0 (IR0) até 7 (IR7).
- **Rotação:** em algumas aplicações, diversos dispositivos que geram interrupções têm a mesma prioridade. Nesse modo, um dispositivo, depois de ser atendido, recebe a menor prioridade no grupo.
- **Máscara especial:** isso permite que o processador iniba interrupções de certos dispositivos.



A interface de periférico programável Intel 82C55A

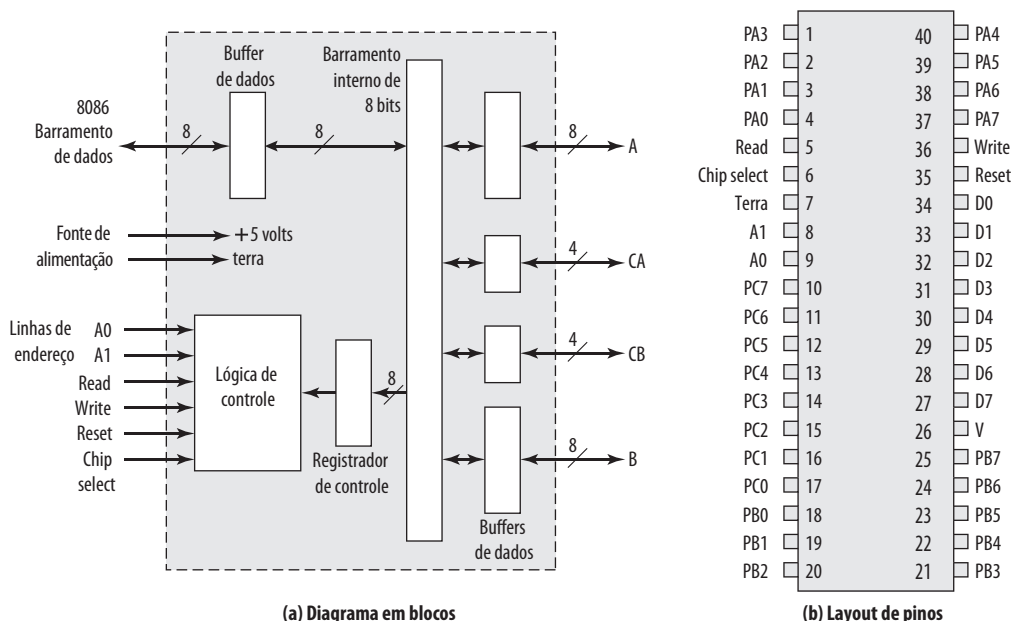
Como um exemplo de um módulo de E/S usado para a E/S programada e E/S controlada por interrupção, consideramos o módulo Intel 82C55A *Programmable Peripheral Interface*. O 82C55A é um módulo de E/S de uso geral

Figura 7.8 Uso do controlador de interrupção 82C59A

em um único chip, projetado para uso com o processador Intel 80386. A Figura 7.9 mostra um diagrama em blocos geral mais a atribuição de pinos para o pacote de 40 pinos em que ele é acomodado.

O lado direito do diagrama em blocos é a interface externa do 82C55A. As 24 linhas de E/S são programáveis pelo 80386 por meio do registrador de controle. O 80386 pode definir o valor do registrador de controle para especificar uma série de modos operacionais e configurações. As 24 linhas são divididas em três grupos de 8 bits (A, B, C). Cada grupo pode funcionar como uma porta de E/S de 8 bits. Além disso, o grupo C é subdividido em grupos de 4 bits (C_A e C_B), que podem ser usados em conjunto com as portas de E/S A e B. Configurado dessa forma, as linhas do grupo C transportam sinais de controle e de estado.

O lado esquerdo do diagrama em blocos é a interface interna do barramento do 80386. Ele inclui um barramento de dados bidirecional de 8 bits (D0 até D7), usado para transferir dados de e para as portas de E/S e transferir informações de controle ao registrador de controle. As duas linhas de endereço especificam uma das três portas de

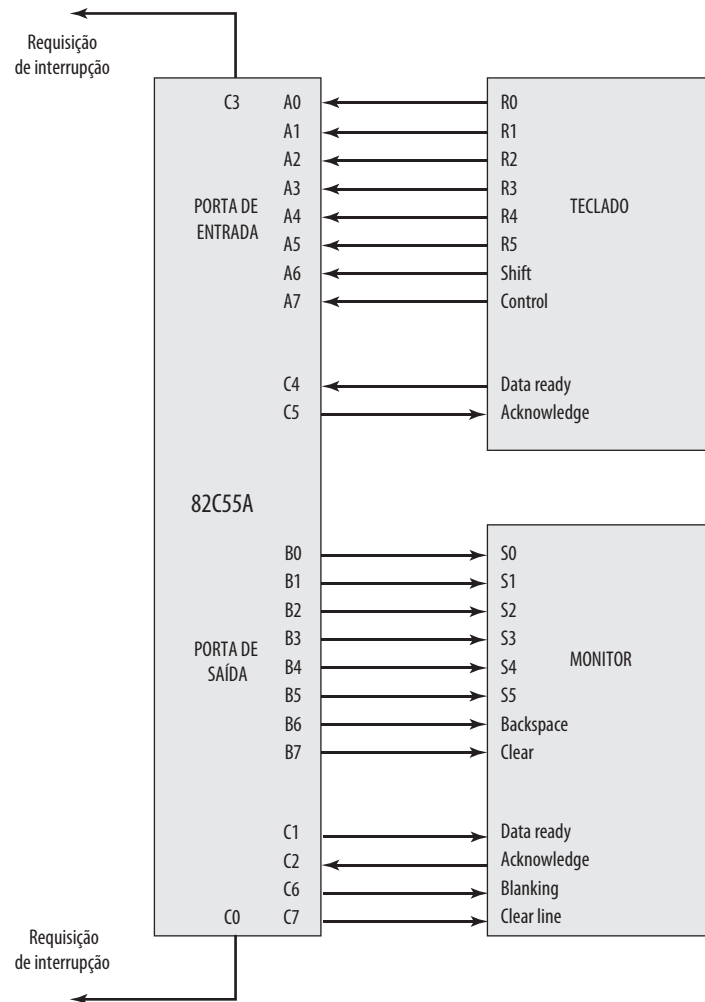
Figura 7.9 O módulo Intel 82C55A *Programmable Peripheral Interface*

E/S ou o registrador de controle. Uma transferência ocorre quando a linha CHIP SELECT é ativada junto com a linha READ ou WRITE. A linha RESET é usada para inicializar o módulo.

O registrador de controle é carregado pelo processador para controlar o modo de operação e definir sinais, se houver. Na operação no Modo 0, os três grupos de oito linhas externas funcionam como três portas de E/S de 8 bits. Cada porta pode ser projetada como entrada ou saída. Caso contrário, os grupos A e B funcionam como portas de E/S, e as linhas do grupo C servem como linhas de controle para A e B. Os sinais de controle têm duas finalidades principais: **handshaking** e requisição de interrupção. O **handshaking** é um mecanismo de temporização simples. Uma linha de controle é usada pelo emissor como uma linha DATA READY, para indicar quando os dados estão presentes nas linhas de dados de E/S. Outra linha é usada pelo receptor como um ACKNOWLEDGE, indicando que os dados foram lidos e as linhas de dados podem ser apagadas. Outra linha pode ser designada como uma linha INTERRUPT REQUEST e ligada de volta ao barramento do sistema.

Como o 82C55A é programável por meio do registrador de controle, ele pode ser usado para controlar diversos dispositivos periféricos simples. A Figura 7.10 ilustra seu uso para controlar um teclado/terminal de vídeo. O teclado oferece 8 bits de entrada. Dois desses bits, SHIFT e CONTROL, possuem significado especial ao programa de tratamento de teclado executado pelo processador. Porém, essa interpretação é transparente ao 82C55A, que simplesmente aceita os 8 bits de dados e os apresenta no barramento de dados do sistema. Duas linhas de controle de **handshaking** são fornecidas para uso com o teclado.

O monitor também é ligado por uma porta de dados de 8 bits. Novamente, dois dos bits possuem significados especiais, que são transparentes ao 82C55A. Além das duas linhas de **handshaking**, duas linhas oferecem funções de controle adicionais.

Figura 7.10 Interface de teclado/monitor para o 82C55A

7.5 Acesso direto à memória

Desvantagens da E/S programada e controlada por interrupção

A E/S controlada por interrupção, embora mais eficiente que a E/S programada, ainda requer a intervenção ativa do processador para transferir dados entre a memória e um módulo de E/S, e quaisquer transferências de dados precisam atravessar um caminho passando pelo processador. Assim, essas duas formas de E/S têm duas desvantagens inerentes:

1. A taxa de transferência de E/S é limitada pela velocidade com a qual o processador pode testar e atender a um dispositivo.
2. O processador fica ocupado no gerenciamento de uma transferência de E/S; diversas instruções precisam ser executadas para cada transferência de E/S (como um exemplo, veja a Figura 7.5).

Existe uma espécie de escolha entre essas duas desvantagens. Considere a transferência de um bloco de dados. Usando a E/S programada simples, o processador é dedicado à tarefa de E/S e pode mover dados em uma taxa

relativamente alta, à custo de não fazer mais nada. A E/S por interrupção libera o processador até certo ponto, mas depende da taxa de transferência de E/S. Apesar disso, os dois métodos possuem um impacto negativo sobre a atividade do processador e a taxa de transferência de E/S.

Quando grandes volumes de dados precisam ser movidos, uma técnica mais eficiente é necessária: acesso direto à memória (DMA).



Função do DMA

DMA envolve um módulo adicional no barramento do sistema. O módulo de DMA (Figura 7.11) é capaz de imitar o processador e, na realidade, assumir o controle do sistema do processador. Ele precisa fazer isso para transferir dados de e para a memória pelo barramento do sistema. Para essa finalidade, o módulo de DMA precisa usar o barramento apenas quando o processador não precisa dele, ou então precisa forçar o processador a suspender a operação temporariamente. Essa última técnica é mais comum e é conhecida como **roubo de ciclo (cycle stealing)**, pois o módulo de DMA efetivamente rouba um ciclo do barramento.

Quando o processador deseja ler ou escrever um bloco de dados, ele envia um comando ao módulo de DMA com as seguintes informações:

- Indicação de uma operação de leitura ou escrita usando a linha de controle de leitura ou escrita entre o processador e o módulo de DMA.
- O endereço do dispositivo de E/S envolvido, comunicado nas linhas de dados.
- O local inicial na memória para ler ou escrever, comunicado nas linhas de dados e armazenado pelo módulo de DMA em seu registrador de endereço.
- O número de palavras a serem lidas ou gravadas, novamente comunicado por meio das linhas de dados e armazenado no registrador contador de dados.

O processador, então, continua com outro trabalho. Ele delegou essa operação de E/S a um módulo de DMA. O módulo de DMA transfere o bloco de dados inteiro, uma palavra de cada vez, diretamente de ou para a memória, sem passar pelo processador. Quando a transferência termina, o módulo de DMA envia um sinal de interrupção ao processador. Assim, o processador é envolvido apenas no início e no final da transferência (Figura 7.4c).

A Figura 7.12 mostra onde, no ciclo de instrução, o processador pode ser suspenso. Em cada caso, o processador é suspenso exatamente antes de precisar usar o barramento. O módulo de DMA, então, transfere uma palavra e

Figura 7.11 Diagrama em blocos típico do DMA

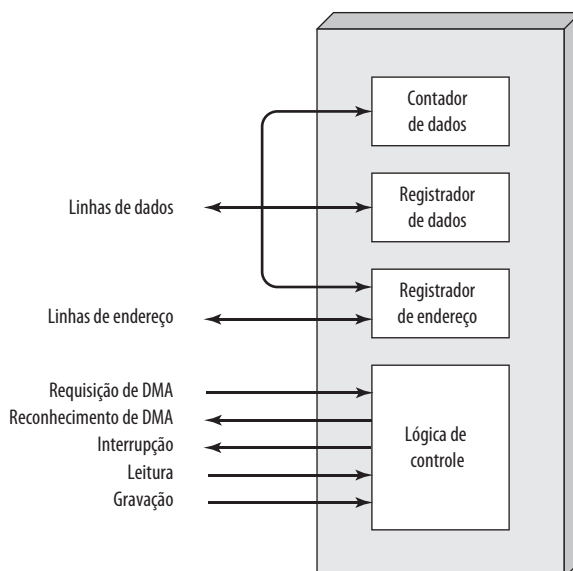
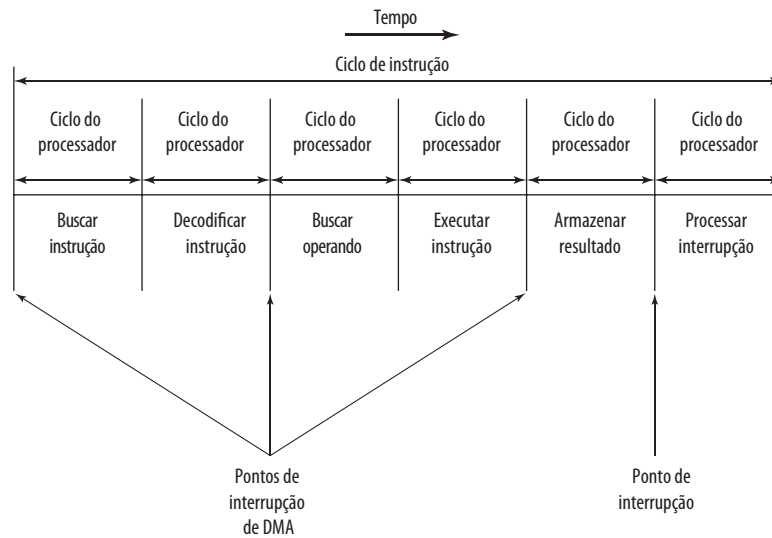


Figura 7.12 DMA e pontos de interrupção durante um ciclo de instrução

retorna o controle ao processador. Observe que isso não é uma interrupção; o processador não salva o contexto e faz algo mais. Em vez disso, o processador é interrompido por um ciclo do barramento. O efeito geral é fazer com que o processador execute mais lentamente. Apesar disso, para uma transferência de E/S de múltiplas palavras, o DMA é muito mais eficiente do que a E/S controlada por interrupção ou programada.

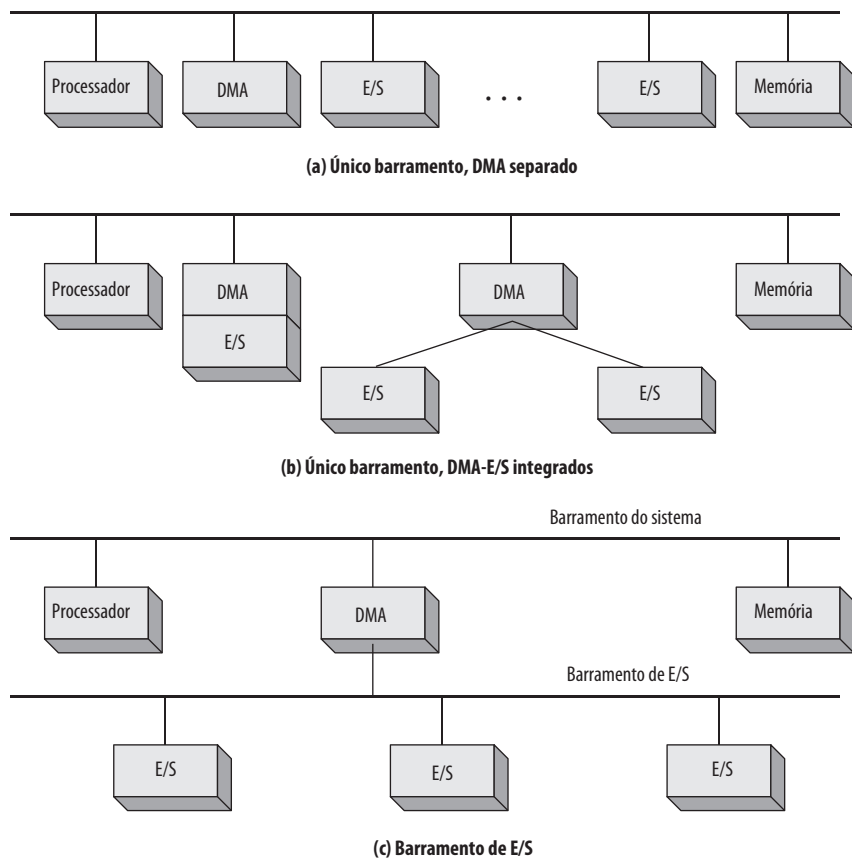
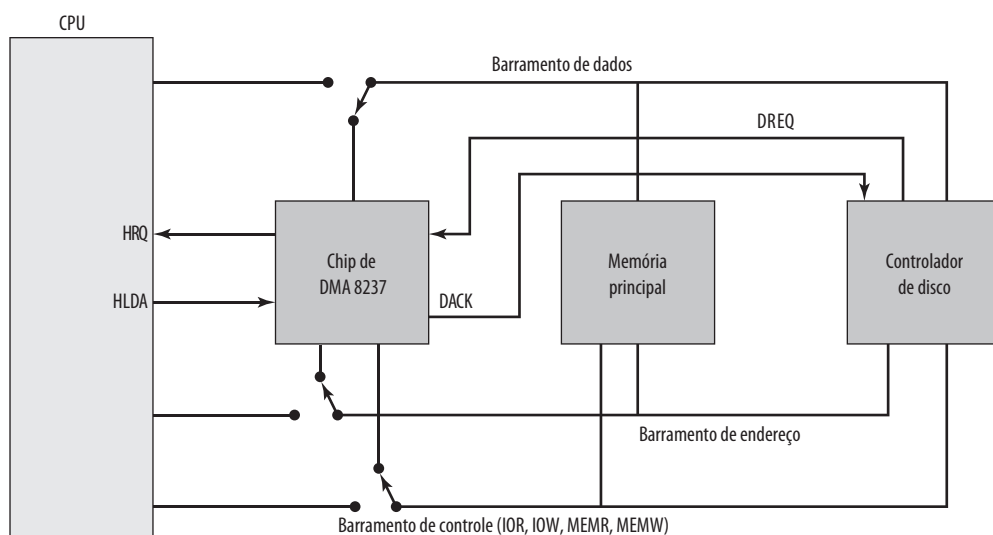
O mecanismo de DMA pode ser configurado de diversas maneiras. Algumas possibilidades aparecem na Figura 7.13. No primeiro exemplo, todos os módulos compartilham o mesmo barramento do sistema. O módulo de DMA, atuando como um processador substituto, utiliza E/S programada para trocar dados entre a memória e um módulo de E/S por meio do módulo de DMA. Essa configuração, embora possa ser pouco dispendiosa, certamente é ineficaz. Assim como a E/S programada controlada pelo processador, cada transferência de uma palavra consome dois ciclos de barramento.

O número de ciclos de barramento exigidos pode ser reduzido substancialmente integrando as funções de DMA e E/S. Como a Figura 7.13b indica, isso significa que existe um caminho entre o módulo de DMA e um ou mais módulos de E/S, que não inclui o barramento do sistema. A lógica de DMA pode realmente fazer parte de um módulo de E/S, ou pode ser um módulo separado que controla um ou mais módulos de E/S. Esse conceito pode ser levado um passo adiante conectando módulos de E/S a um módulo de DMA, usando um barramento de E/S (Figura 7.13c). Isso reduz o número de interfaces de E/S no módulo de DMA a um e oferece uma configuração facilmente expansível. Nesses dois casos (Figuras 7.13b e c), o barramento do sistema que o módulo de DMA compartilha com o processador e a memória é usado pelo módulo de DMA somente para trocar dados com a memória. A troca de dados entre os módulos de DMA e E/S ocorre fora do barramento do sistema.



Controlador de DMA Intel 8237A

O controlador de DMA Intel 8237A realiza a interface com a família de processadores 80x86 e com uma memória DRAM, para oferecer uma capacidade de DMA. A Figura 7.14 indica o local do módulo de DMA. Quando o módulo de DMA precisa usar os barramentos do sistema (dados, endereço e controle) para transferir dados, ele envia um sinal denominado HOLD ao processador. O processador responde com o sinal HLDA (*hold acknowledge*), indicando que o módulo de DMA pode usar os barramentos. Por exemplo, se o módulo de DMA tiver que transferir um bloco de dados da memória ao disco, ele fará o seguinte:

Figura 7.13 Configurações de DMA alternativas**Figura 7.14** Uso do barramento do sistema pelo controlador de DMA 8237

DACK = DMA acknowledge (reconhecimento de DMA)
 DREQ = DMA request (requisição de DMA)
 HLDA = HOLD acknowledge (reconhecimento de HOLD)
 HRQ = HOLD request (requisição de HOLD)

1. O dispositivo periférico (como o controlador de disco) requisitará o serviço de DMA levantando o sinal DREQ (requisição de DMA).
2. O DMA levantará sua linha HRQ (requisição de HOLD), sinalizando à CPU através de seu pino HOLD que ele precisa usar os barramentos.
3. A CPU terminará o ciclo de barramento atual (não necessariamente a instrução atual) e responderá à solicitação de DMA levantando sua linha HDLA (confirmação de HOLD), dizendo assim ao DMA 8237 que ele pode seguir em frente e usar os barramentos para realizar sua tarefa. A linha de HOLD precisa permanecer ativa enquanto o DMA estiver realizando sua tarefa.
4. O DMA ativará a linha DACK (confirmação de DMA), que diz ao dispositivo periférico que ele começará a transferir os dados.
5. O DMA começa a transferir os dados da memória para o periférico, colocando o endereço do primeiro byte do bloco no barramento de endereço e ativando MEMR, lendo assim o byte da memória para o barramento de dados; depois, ele ativa IOW para escrevê-lo no periférico. Em seguida, o DMA decrementa o contador e incrementa o ponteiro de endereço, repetindo esse processo até que a contagem chegue a zero e a tarefa esteja encerrada.
6. Depois que o DMA terminar seu trabalho, ele desativará HRQ, sinalizando à CPU que ela pode retomar o controle de seus barramentos.

Enquanto o DMA está usando os barramentos para transferir dados, o processador fica ocioso. De modo semelhante, quando o processador está usando o barramento, o DMA fica ocioso. O DMA 8237 é conhecido como um controlador de DMA *flutuante*. Isso significa que os dados movidos de um local para outro não passam pelo chip de DMA e não são armazenados nele. Portanto, o DMA só pode transferir dados entre uma porta de E/S e um endereço de memória, mas não entre duas portas de E/S ou dois locais de memória. Porém, conforme explicamos mais adiante, o chip de DMA pode realizar uma transferência de memória a memória através de um registrador.

O 8237 contém quatro canais de DMA, que podem ser programados independentemente, e qualquer um deles pode estar ativo a qualquer momento. Esses canais são numerados com 0, 1, 2 e 3.

O 8237 tem um conjunto de cinco registradores de controle/comando para programar e controlar a operação de DMA por um de seus canais (Tabela 7.2):

Tabela 7.2 Registradores do Intel 8237A

Bit	Command	Estado	Mode	Single Mask	All Mask
D0	H/D memória para memória	Canal 0 atingiu CF	Seleção de canal	Seleciona bit de máscara do canal	Apaga/marca bit de máscara do canal 0
D1	H/D hold de endereço do canal 0	Canal 1 atingiu CF			Apaga/marca bit de máscara do canal 1
D2	H/D controlador	Canal 2 atingiu CF	Verificar/escrever/ler transferência	Apaga/marca bit de máscara	Apaga/marca bit de máscara do canal 2
D3	Temporização normal/comprimida	Canal 3 atingiu CF		Não usado	Apaga/marca bit de máscara do canal 3
D4	Prioridade fixa/rotativa	Requisição do canal 0	H/D autoinicialização		Não usado
D5	Seleção de escrita adiada/estendida	Requisição do canal 0	Seleção de incremento/decremento de endereço		
D6	Percepção de DREQ ativo alto/baixo	Requisição do canal 0			
D7	Percepção de DACK ativo alto/baixo	Requisição do canal 0	Seleção de modo <i>demand/single/block/cascade</i>		

H/D = Habilita/desabilita

CF = Contagem final

- **Command:** o processador carrega esse registrador para controlar a operação do DMA. D0 habilita uma transferência de memória para memória, em que o canal 0 é usado para transferir um byte para um registrador temporário do 8237 e o canal 1 é usado para transferir o byte do registrador para a memória. Quando a transferência de memória para memória está habilitada, D1 pode ser usado para desativar o incremento/decremento no canal 0, de modo que um valor fixo pode ser escrito em um bloco de memória. D2 habilita ou desabilita o DMA.
- **Status:** o processador lê esse registrador para determinar o estado do DMA. Os bits D0-D3 são usados para indicar se os canais 0-3 atingiram sua CF (contagem final). Os bits D4-D7 são usados pelo processador para determinar se algum canal possui uma requisição de DMA pendente.
- **Mode:** o processador define esse registrador para determinar o modo de operação do DMA. Os bits D0 e D1 são usados para selecionar um canal. Os outros bits selecionam diversos modos de operação para o canal selecionado. Os bits D2 e D3 determinam se a transferência é de um dispositivo de E/S para a memória (escrita) ou da memória para a E/S (leitura), ou uma operação de verificação. Se D4 estiver marcado, então o registrador de endereço de memória e o registrador contador são recarregados com seus valores originais ao final de uma transferência de dados por DMA. Os bits D6 e D7 determinam o modo como o 8237 é utilizado. No modo *single*, um único byte de dados é transferido. Os modos *block* e *demand* são usados para uma transferência em bloco, com o modo *demand* permitindo o término prematuro da transferência. O modo *cascade* permite que vários 8237s sejam dispostos em cascata, expandindo o número de canais para mais de 4.
- **Single Mask:** o processador define esse registrador. Os bits D0 e D1 selecionam o canal. O bit D2 apaga ou define o bit de máscara para esse canal. É através desse registrador que a entrada DREQ de um canal específico pode ser mascarada (desabilitada) ou desmascarada (habilitada). Enquanto o registrador *command* pode ser usado para desabilitar o chip de DMA inteiro, o registrador *single mask* permite que o programador desabilite ou habilite um canal específico.
- **All Mask:** esse registrador é semelhante ao registrador *single mask*, exceto que todos os canais podem ser mascarados ou desmascarados com uma operação de escrita.

Além disso, o 8237A tem oito registradores de dados: um registrador de endereço de memória e um registrador de contagem para cada canal. O processador define esses registradores para indicar o local da memória principal a ser afetado pelas transferências.



7.6 Canais e processadores de E/S



A evolução da função de E/S

Com a evolução dos sistemas de computação, tem havido um crescimento no padrão de complexidade e sofisticação dos componentes individuais. Em nenhum outro lugar isso é mais evidente do que na função de E/S. Já vimos parte dessa evolução. As etapas dessa evolução podem ser resumidas da seguinte forma:

1. A CPU controla diretamente o dispositivo periférico. Isso é visto em dispositivos simples controlados por microprocessador.
2. Um controlador ou módulo de E/S é acrescentado. A CPU usa a E/S programada sem interrupções. Com essa etapa, a CPU fica por fora dos detalhes específicos das interfaces do dispositivo externo.
3. A mesma configuração da etapa 2 é utilizada, mas agora as interrupções são empregadas. A CPU não precisa gastar tempo esperando que uma operação de E/S seja realizada, aumentando assim sua eficiência.
4. O módulo de E/S recebe acesso direto à memória, por meio de DMA. Ele agora pode mover um bloco de dados de ou para a memória sem envolver a CPU, exceto no início e no final da transferência.
5. O módulo de E/S é aprimorado para se tornar um processador por conta própria, com um conjunto especializado de instruções, ajustado para E/S. A CPU direciona o processador de E/S a executar um programa de E/S armazenado na memória. O processador de E/S busca e executa essas instruções sem intervenção da CPU. Isso permite que a CPU especifique uma sequência de atividades de E/S e seja interrompida somente quando a sequência inteira tiver sido executada.
6. O módulo de E/S tem uma memória local própria e, de fato, é um computador separado. Com essa arquitetura, um grande conjunto de dispositivos de E/S pode ser controlado, com o mínimo de envolvimento da CPU. Um uso comum para essa arquitetura tem sido no controle da comunicação com terminais interativos. O processador de E/S cuida da maior parte das tarefas envolvidas no controle dos terminais.

Enquanto se prossegue nesse caminho de evolução, cada vez mais a função de E/S é realizada sem envolvimento da CPU. A CPU fica cada vez mais livre das tarefas relacionadas a E/S, melhorando o desempenho. Com as duas últimas etapas (5-6), ocorre uma grande mudança com a introdução do conceito de um módulo de E/S capaz de executar um programa. Para a etapa 5, o módulo de E/S normalmente é conhecido como um **canal de E/S**. Para a etapa 6, o termo **processador de E/S** normalmente é utilizado. Contudo, os dois termos ocasionalmente são aplicados às duas situações. No texto seguinte, usaremos o termo **canal de E/S**.

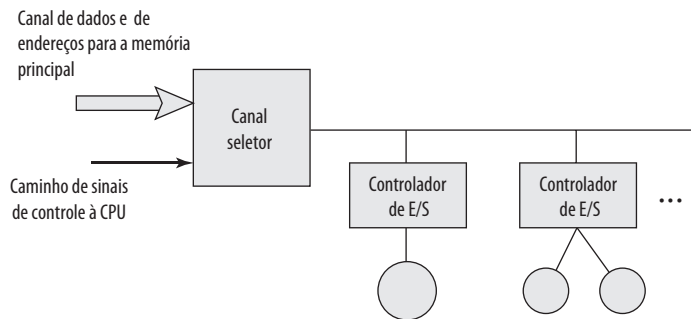


Características dos canais de E/S

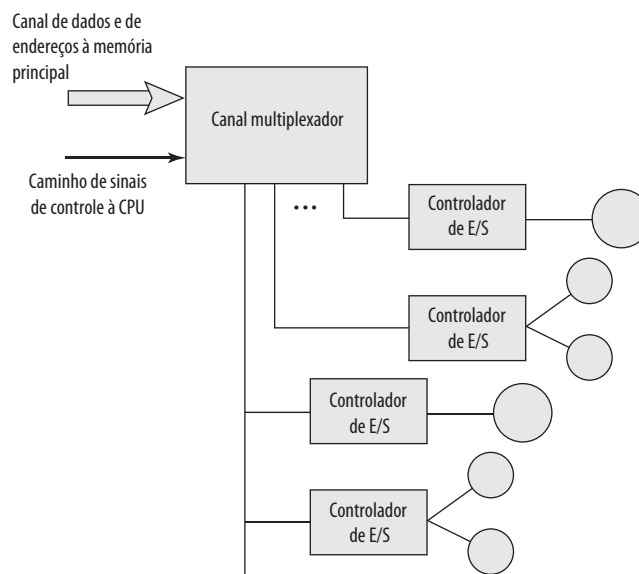
O canal de E/S representa uma extensão do conceito de DMA. Um canal de E/S tem a capacidade de executar instruções de E/S, o que lhe oferece um controle completo sobre as operações de E/S. Em um sistema de computação com esses dispositivos, a CPU não executa instruções de E/S. Essas instruções são armazenadas na memória principal para serem executadas por um processador de uso específico no próprio canal de E/S. Assim, a CPU inicia uma transferência de E/S instruindo o canal de E/S a executar um programa na memória. O programa especificará o dispositivo ou dispositivos, a área ou as áreas da memória para armazenamento, prioridade e ações a serem tomadas para certas condições de erro. O canal de E/S segue essas instruções e controla a transferência de dados.

Dois tipos de canais de E/S são comuns, conforme ilustramos na Figura 7.15. Um **canal seletor** controla múltiplos dispositivos de alta velocidade e, a qualquer momento, é dedicado à transferência de dados com um desses

Figura 7.15 Arquitetura do canal de E/S



(a) Seletor



(b) Multiplexador

dispositivos. Assim, o canal de E/S seleciona um dispositivo e efetua a transferência de dados. Cada dispositivo, ou pequeno grupo de dispositivos, é tratado por um **controlador**, ou módulo de E/S, que é semelhante aos módulos de E/S que discutimos até aqui. Assim, o canal de E/S atua no lugar da CPU para controlar esses controladores de E/S. Um **canal multiplexador** pode tratar da E/S com vários dispositivos ao mesmo tempo. Para dispositivos de baixa velocidade, um **multiplexador de byte** aceita ou transmite caracteres o mais rápido possível a diversos dispositivos. Por exemplo, o fluxo de caracteres resultante de três dispositivos com diferentes velocidades e fluxos individuais A1A2A3A4..., B1B2B3B4... e C1C2C3C4... poderia ser A1B1C1A2C2A3B2C3A4, e assim por diante. Para dispositivos de alta velocidade, um **multiplexador de bloco** intercala os blocos de dados de vários dispositivos.

7.7 A interface externa: FireWire e InfiniBand

Tipos de interfaces

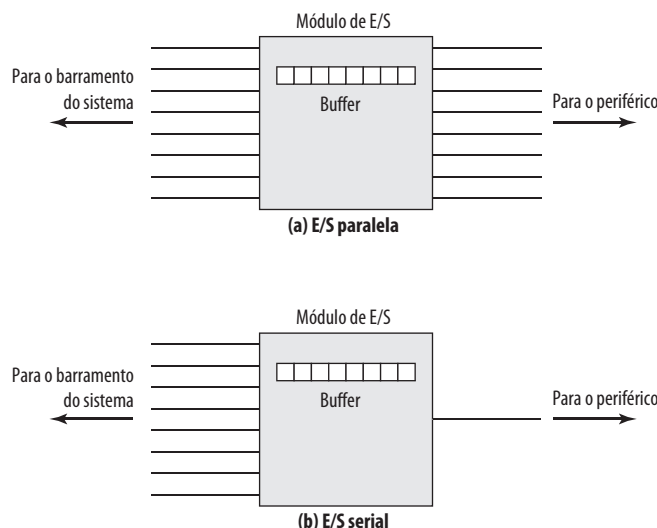
A interface para um periférico a partir de um módulo de E/S precisa ser ajustada à natureza e à operação do periférico. Uma característica importante da interface é se ela é serial ou paralela (Figura 7.16). Em uma **interface paralela**, existem múltiplas linhas conectando o módulo de E/S e o periférico, e diversos bits são transferidos simultaneamente, assim como todos os bits de uma palavra são transferidos simultaneamente pelo barramento de dados. Em uma **interface serial**, há apenas uma linha usada para transmitir dados, e os bits precisam ser transmitidos um de cada vez. Uma interface paralela tradicionalmente tem sido usada para periféricos de mais alta velocidade, como fita e disco, enquanto a interface serial tradicionalmente tem sido usada para impressoras e terminais. Com uma nova geração de interfaces seriais de alta velocidade, as interfaces paralelas estão se tornando muito menos comuns.

Nos dois casos, o módulo de E/S precisa tomar parte da interação com o periférico. Em termos gerais, a interação para uma operação de escrita é o seguinte:

1. O módulo de E/S envia um sinal de controle requisitando permissão para enviar dados.
2. O periférico reconhece a requisição.
3. O módulo de E/S transfere dados (uma palavra ou um bloco, dependendo do periférico).
4. O periférico confirma o recebimento dos dados.

Uma operação de leitura prossegue de forma semelhante.

Figura 7.16 E/S paralela e serial



A chave para a operação de um módulo de E/S é um buffer interno que pode armazenar os dados que estão sendo passados entre o periférico e o restante do sistema. Esse buffer permite que o módulo de E/S compense as diferenças na velocidade entre o barramento do sistema e suas linhas externas.



Configurações ponto a ponto e multiponto

A conexão entre um módulo de E/S em um sistema de computação e os dispositivos externos pode ser ponto a ponto ou multiponto. Uma interface ponto a ponto oferece uma linha dedicada entre o módulo de E/S e o dispositivo externo. Em sistemas pequenos (PCs, estações de trabalho), as conexões ponto a ponto típicas são usadas para o teclado, impressora e modem externo. Um exemplo típico desse tipo de interface é a especificação EIA-232 (veja uma descrição em Stallings, 2009^a).

De importância cada vez maior são as interfaces externas multiponto, usadas para dar suporte a dispositivos externos de armazenamento em massa (unidades de disco e fita) e dispositivos de multimídia (CD-ROMs, vídeo, áudio). Essas interfaces multiponto são, de fato, barramentos externos, e exibem o mesmo tipo de lógica dos barramentos discutidos no Capítulo 3. Nesta seção, examinamos dois exemplos importantes: FireWire e InfiniBand.



Barramento serial FireWire

Com velocidades do processador alcançando a faixa dos gigahertz e dispositivos de armazenamento mantendo múltiplos gigabits, as demandas de E/S para computadores pessoais, estações de trabalho e servidores são formidáveis. Mesmo assim, as tecnologias de canal de E/S de alta velocidade que foram desenvolvidas para sistemas de mainframe e supercomputador são muito caras e volumosas para serem usadas nesses sistemas menores. Consequentemente, tem havido grande interesse no desenvolvimento de uma alternativa de alta velocidade para a *small computer system interface* (SCSI) e outras interfaces de E/S para sistemas pequenos. O resultado é o padrão IEEE 1394, para um barramento serial de alto desempenho (*high performance serial bus*), normalmente conhecido como FireWire.

FireWire tem diversas vantagens em relação às interfaces de E/S mais antigas. Ela tem velocidade mais alta, baixo custo e é fácil de se implementar. De fato, FireWire é favorável não apenas para sistemas de computação, mas também para produtos eletrônicos para o consumidor, como câmeras digitais, aparelhos de reprodução e gravação de DVD, e televisores. Nesses produtos, o FireWire é usado para transportar imagens de vídeo, que cada vez mais vêm de fontes digitalizadas.

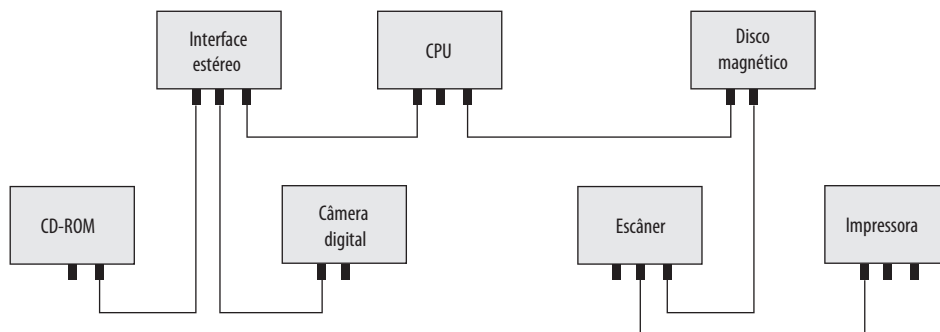
Um dos pontos fortes da interface FireWire é que ela usa a transmissão serial (um bit de cada vez) ao invés da paralela. As interfaces paralelas, como SCSI, exigem mais fios, o que significa cabos mais caros e mais grossos, e conectores maiores e mais caros, com mais pinos para entortar ou quebrar. Um cabo com mais fios exige blindagem para impedir interferência elétrica entre os fios. Além disso, com uma interface paralela, o sincronismo entre os fios torna-se um requisito, um problema que piora com o aumento da extensão do cabo.

Além disso, os computadores estão se tornando fisicamente menores, mesmo enquanto aumentam seus requisitos de potência de computação e E/S. Computadores portáteis e de bolso têm pouco espaço para conectores, embora precisem de altas taxas de dados para lidar com imagens e vídeo.

A intenção da interface FireWire é oferecer uma única interface de E/S com um único conector, que pode tratar de diversos dispositivos através de uma única porta, de modo que o mouse, impressora a laser, unidade de disco externa, som e conexões de rede local possam ser substituídos por esse único conector.

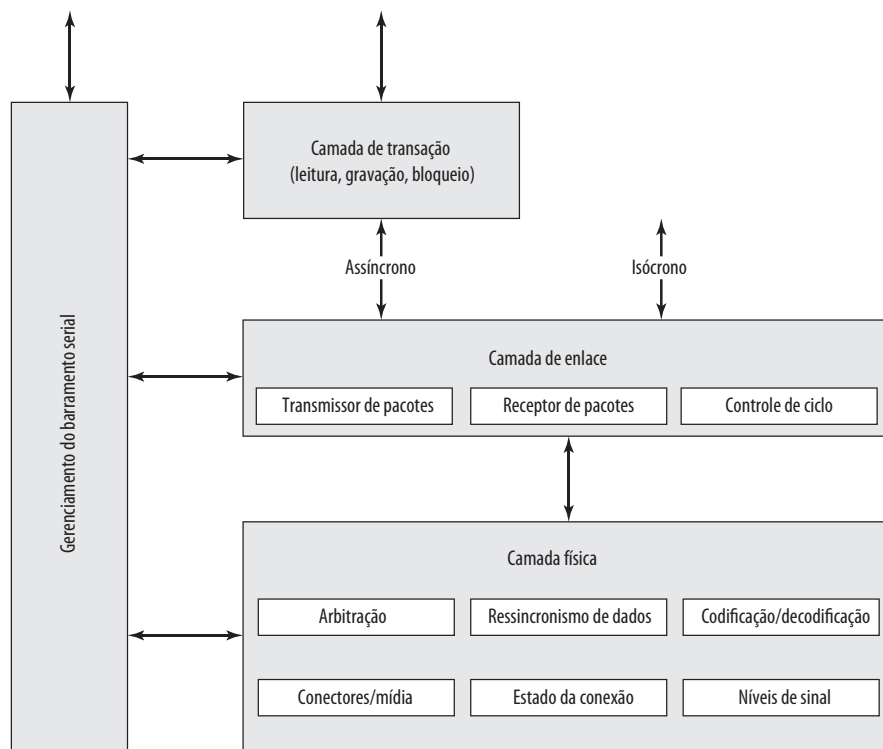
CONFIGURAÇÕES DE FIREWIRE O barramento FireWire utiliza uma configuração Daisy chain, com até 63 dispositivos conectados a partir de uma única porta. Além do mais, até 1 022 barramentos FireWire podem ser interconectados usando **pontes**, permitindo que um sistema aceite tantos periféricos quantos forem necessários.

O FireWire permite o que é conhecido como *conexão a quente* (*hot plugging*), que significa que é possível conectar e desconectar periféricos sem ter que desligar o sistema de computação ou reconfigurar o sistema. Além disso, FireWire permite configuração automática; não é necessário definir manualmente IDs de dispositivo ou se preocupar com a posição relativa dos dispositivos. A Figura 7.17 mostra uma configuração FireWire simples. Com FireWire, não existem terminações, e o sistema executa automaticamente uma função de configuração para atribuir endereços. Observe também que um barramento FireWire não precisa ser uma Daisy chain estrita. Em vez disso, é possível usar uma configuração estruturada em forma de árvore.

Figura 7.17 Configuração FireWire simples

Um recurso importante do padrão FireWire é que ele especifica um conjunto de três camadas de protocolos para padronizar o modo como o sistema *principal* interage com os dispositivos periféricos pelo barramento serial. A Figura 7.18 ilustra essa pilha. As três camadas da pilha são as seguintes:

- **Camada física:** define os meios de transmissão que são permitidos sob FireWire e as características elétrica e de sinalização de cada um.
- **Camada de enlace:** descreve a transmissão de dados nos pacotes.
- **Camada de transação:** define um protocolo de requisição-resposta que esconde das aplicações os detalhes da camada inferior do FireWire.

Figura 7.18 Pilha de protocolos FireWire

CAMADA FÍSICA A camada física do FireWire especifica vários meios de transmissão alternativos e seus conectores, com diferentes propriedades físicas e de transmissão de dados. Taxas de dados de 25 a 3 200 Mbps são definidas. A camada física converte dados binários em sinais elétricos de vários meios físicos. Essa camada também oferece serviço de arbitragem que garante que somente um dispositivo de cada vez transmitirá dados.

Duas formas de arbitragem são oferecidas pelo FireWire. A forma mais simples é baseada no arranjo, estruturado em árvores dos nós em um barramento FireWire, mencionado anteriormente. Um caso especial dessa estrutura é uma Daisy chain linear. A camada física contém a lógica que permite que todos os dispositivos conectados se configurem de modo que um nó seja designado como a raiz da árvore e outros nós sejam organizados em um relacionamento de pai/filho, formando a topologia de árvore. Quando essa configuração é estabelecida, o nó raiz atua como um árbitro central, e processa solicitações para acesso ao barramento no padrão primeiro a chegar, primeiro a ser atendido. No caso de requisições simultâneas, o nó com a prioridade natural mais alta recebe o acesso. A prioridade natural é determinada por qual nó concorrente é o mais próximo da raiz e, entre aqueles com a mesma distância da raiz, qual tem o menor número de ID.

O método de arbitragem mencionado é suplementado por duas funções adicionais: arbitragem imparcial (*fairness arbitration*) e arbitragem urgente. Com a arbitragem imparcial, o tempo no barramento é organizado em **intervalos imparciais**. No início de um intervalo, cada nó define um flag `arbitration_enable`. Durante o intervalo, cada nó pode competir pelo acesso ao barramento. Quando um nó tiver ganho acesso ao barramento, ele reinicia seu flag `arbitration_enable` e não pode mais competir pelo acesso imparcial durante esse intervalo. Esse esquema torna a arbitragem mais justa, pois impede que um ou mais dispositivos de alta prioridade muito atarefados monopolizem o barramento.

Além do esquema imparcial, alguns dispositivos podem ser configurados como tendo uma prioridade **urgente**. Esses nós podem ganhar o controle do barramento várias vezes durante um intervalo imparcial. Basicamente, um contador é usado em cada nó de alta prioridade, que permite que os nós de alta prioridade controlem 75% do tempo disponível no barramento. Para cada pacote que é transmitido como não urgente, três pacotes podem ser transmitidos como urgentes.

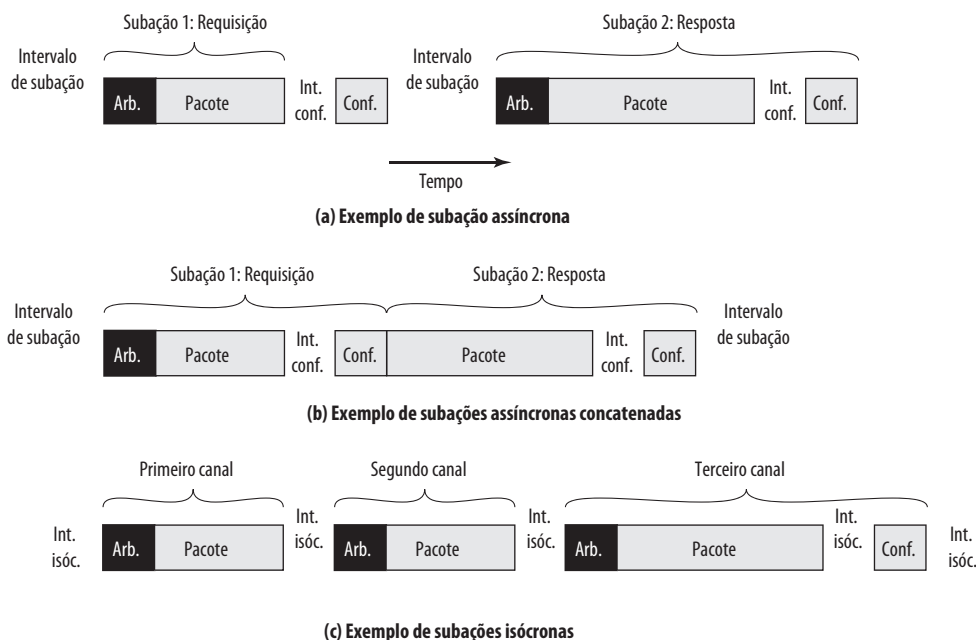
CAMADA DE ENLACE A camada de enlace define a transmissão de dados na forma de pacotes. Dois tipos de transmissão são aceitos:

- **Assíncrono:** uma quantidade variável de dados e vários bytes de informações da camada de transação são transferidos como um pacote para um endereço explícito e uma confirmação é retornada.
- **Isócrono:** uma quantidade variável de dados é transferida em uma sequência de pacotes de tamanho fixo, transmitidos em intervalos regulares. Essa forma de transmissão utiliza endereçamento simplificado, sem reconhecimento.

A transmissão assíncrona é usada por dados que não possuem requisitos fixos de taxa de dados. Os esquemas de arbitragem imparcial e arbitragem urgente podem ser usados para a transmissão assíncrona. O método padrão é a arbitragem imparcial. Os dispositivos que desejam uma parte substancial da capacidade do barramento ou que tenham fortes requisitos de latência utilizam o método de arbitragem urgente. Por exemplo, um nó de coleta de dados em tempo real de alta velocidade pode usar a arbitragem urgente quando os **buffers** de dados críticos estiverem acima da metade de sua capacidade total.

A Figura 7.19a representa uma transação assíncrona típica. O processo de entrega de um único pacote é chamado de subação. A subação consiste em cinco períodos:

- **Sequência de arbitragem:** essa é a troca de sinais exigida para dar a um dispositivo o controle do barramento.
- **Transmissão de pacote:** cada pacote inclui um cabeçalho contendo as IDs de origem e de destino. O cabeçalho também contém informações de tipo de pacote, uma soma de verificação de CRC (*cyclic redundancy check*), e informações de parâmetro para o tipo de pacote específico. Um pacote também pode incluir um bloco de dados consistindo em dados do usuário e outro CRC.
- **Intervalo de confirmação:** esse é o atraso de tempo para que o destino receba e decodifique um pacote e gere um reconhecimento.
- **Confirmação:** o destinatário do pacote retorna um pacote de reconhecimento com um código indicando a ação tomada por ele.
- **Intervalo de subação:** esse é um período ocioso imposto para garantir que outros nós no barramento não comecem a arbitrar antes que o pacote de confirmação tenha sido transmitido.

Figura 7.19 Sub-ações do FireWire

No momento em que a confirmação é enviada, o nó que está confirmando está no controle do barramento. Portanto, se a troca for uma interação de requisição/resposta entre dois nós, então o nó que está respondendo pode imediatamente transmitir o pacote de resposta sem passar por uma sequência de arbitragem (Figura 7.19b).

Para dispositivos que regularmente geram ou consomem dados, como som ou vídeo digital, o acesso isócrono é permitido. Esse método garante que os dados possam ser entregues dentro de uma latência especificada, com uma taxa de dados garantida.

Para acomodar uma carga de tráfego mista de fontes de dados isócronas e assíncronas, um nó é designado como **mestre de ciclo**. Periodicamente, o mestre de ciclo emite um pacote `cycle_start`. Este sinaliza a todos os outros nós, avisando que um ciclo isócrono foi iniciado. Durante esse ciclo, somente os pacotes isócronos podem ser enviados (Figura 7.19c). Cada origem de dados isócrona compete pelo acesso ao barramento. O nó vencedor imediatamente transmite um pacote. Não existe confirmação para esse pacote, e por isso outras fontes de dados isócronas imediatamente disputam o barramento após o pacote isócrono anterior ser transmitido. O resultado é que existe um pequeno intervalo entre a transmissão de um pacote e o período de arbitragem para o próximo pacote, ditado por atrasos no barramento. Esse atraso, conhecido como intervalo isócrono, é menor do que um intervalo de subação.

Após todas as fontes isócronas terem sido transmitidas, o barramento permanecerá ocioso por tempo suficiente para que ocorra um intervalo de subação. Esse é o sinal para as fontes assíncronas, avisando que elas agora podem competir pelo acesso ao barramento. As fontes assíncronas podem então usar o barramento até o início do próximo ciclo isócrono.

Os pacotes isócronos são rotulados com números de canal de 8 bits, que foram previamente atribuídos por uma interação entre os dois nós que devem trocar dados isócronos. O cabeçalho, que é mais curto que aquele para pacotes assíncronos, também inclui um campo de tamanho de dados e um CRC de cabeçalho.



InfiniBand

InfiniBand é uma especificação de E/S recente, voltada para o mercado de servidores de ponta.³ A primeira versão da especificação foi lançada em início de 2001, e tem atraído diversos fornecedores. O padrão descreve

³ InfiniBand é o resultado da união de dois projetos concorrentes: *Future I/O* (com o apoio da Cisco, HP, Compaq e IBM) e *Next Generation I/O* (desenvolvido pela Intel e com o apoio de diversas outras empresas).

uma arquitetura e especificações para o fluxo de dados entre os processadores e dispositivos de E/S inteligentes. InfiniBand tornou-se uma interface popular para redes de armazenamento e outras configurações de armazenamento grandes. Basicamente, o InfiniBand permite que servidores, armazenamento remoto e outros dispositivos de rede sejam conectados em uma fábrica central de comutadores (*switches*) e links. A arquitetura baseada em comutador pode conectar até 64 000 servidores, sistemas de armazenamento e dispositivos de rede.

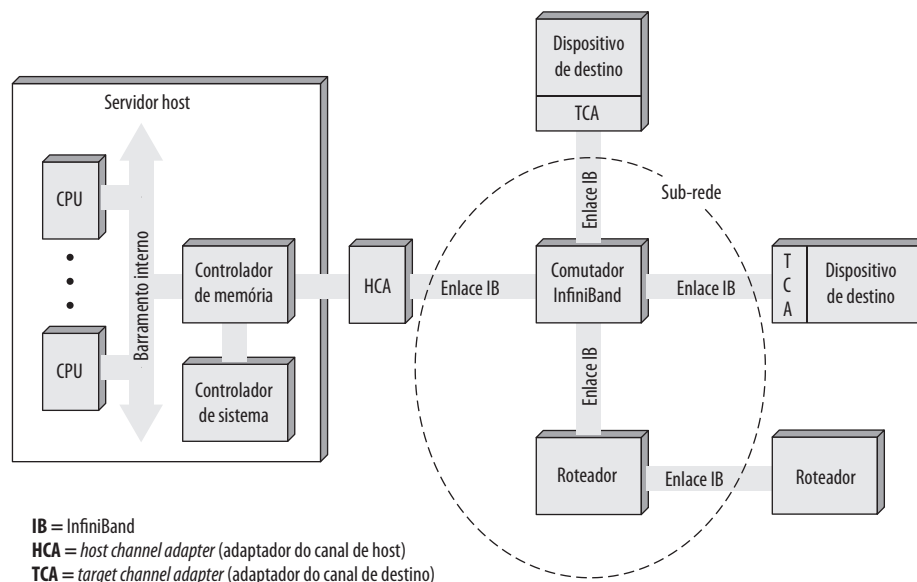
ARQUITETURA INFINIBAND Embora PCI seja um método de interconexão confiável e continue a oferecer velocidades cada vez maiores, de até 4 Gbps, essa é uma arquitetura limitada em comparação com InfiniBand. Com o InfiniBand, não é preciso ter o hardware de interface de E/S básico dentro do chassi do servidor. Com InfiniBand, armazenamento remoto, redes e conexões entre servidores são realizados conectando-se todos os dispositivos a uma estrutura central de comutadores (*switches*) e conexões. A remoção da E/S do chassi do servidor permite maior densidade de servidores e possibilita uma central de dados mais flexível e expansível, visto que os nós independentes podem ser acrescentados conforme a necessidade.

Diferente do PCI, que mede as distâncias a partir da placa mãe da CPU em centímetros, o projeto de canal do InfiniBand permite que os dispositivos de E/S sejam colocados a até 17 metros de distância do servidor usando cobre, até 300 m usando fibra óptica multimodo, e até 10 km com fibra óptica de modo único. Taxas de transmissão de até 30 Gbps podem ser alcançadas.

A Figura 7.20 ilustra a arquitetura InfiniBand. Os principais elementos são os seguintes:

- **Host channel adapter (HCA — adaptador do canal do host):** Em vez de uma série de *slots* PCI, um servidor típico precisa de uma única interface para um HCA, que liga o servidor a um comutador InfiniBand. O HCA se conecta ao servidor em um controlador de memória, que tem acesso ao barramento do sistema e controla o tráfego entre o processador e a memória e entre o HCA e a memória. O HCA usa acesso direto à memória (DMA) para ler e escrever na memória.
- **Target channel adapter (TCA — adaptador do canal de destino):** um TCA é usado para conectar sistemas de armazenamento, roteadores e outros dispositivos periféricos a um comutador InfiniBand.
- **Comutador InfiniBand:** um comutador oferece conexões físicas ponto a ponto para uma série de dispositivos e direciona o tráfego de uma conexão para outra. Os servidores e dispositivos se comunicam por seus adaptadores, através do comutador. A inteligência do comutador gerencia as ligações sem interromper a operação dos servidores.
- **Conexões:** a conexão entre um comutador e um adaptador de canal, ou entre dois comutadores.
- **Sub-rede:** uma sub-rede consiste em um ou mais comutadores interconectados mais os links que conectam outros dispositivos a esses comutadores. A Figura 7.20 mostra uma sub-rede com um único co-

Figura 7.20 Fábrica de comutadores InfiniBand



mutador, porém sub-redes mais complexas são necessárias quando muitos dispositivos tiverem que ser interconectados. As sub-redes permitem que os administradores confinem transmissões de **broadcast** e **multicast** dentro da sub-rede.

- **Roteador:** conecta sub-redes InfiniBand, ou conecta um comutador InfiniBand a uma rede, como uma rede local, uma rede remota ou uma rede de armazenamento.

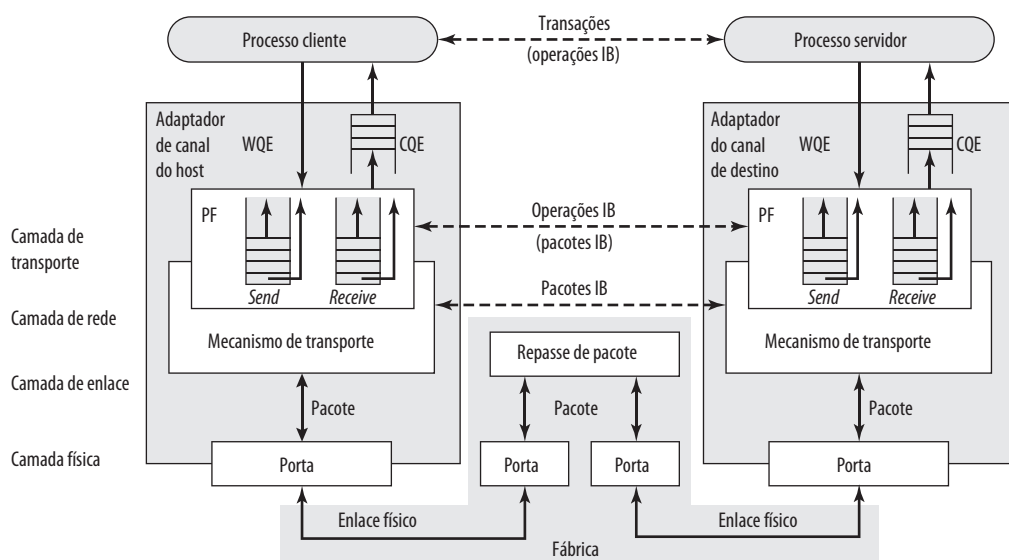
Os adaptadores de canal são dispositivos inteligentes que tratam de todas as funções de E/S sem a necessidade de interromper o processador do servidor. Por exemplo, existe um protocolo de controle pelo qual um comutador descobre todos os TCAs e HCAs na estrutura e atribui endereços lógicos a cada um deles. Isso é feito sem envolvimento do processador.

O comutador InfiniBand temporariamente abre canais entre o processador e os dispositivos com os quais está se comunicando. Os dispositivos não precisam compartilhar a capacidade de um canal, como acontece com um projeto baseado em barramento, como PCI, que exige que os dispositivos disputem o acesso ao processador. Dispositivos adicionais são acrescentados à configuração conectando o TCA de cada dispositivo ao comutador.

OPERAÇÃO DO INFINIBAND Cada enlace físico entre um comutador e uma interface conectada (HCA ou TCA) pode aceitar até 16 canais lógicos, denominados **pistas virtuais**. Uma pista é reservada para o gerenciamento da estrutura e as outras pistas para transporte de dados. Os dados são enviados na forma de um fluxo de pacotes, com cada pacote contendo alguma parte do total de dados a ser transferido, mais informações de endereçamento e controle. Assim, protocolos de comunicações são usados para gerenciar a transferência de dados. Uma pista virtual é dedicada temporariamente à transferência de dados de um nó extremo a outro pela estrutura InfiniBand. O comutador InfiniBand mapeia o tráfego de uma pista de entrada para uma pista de saída para rotear os dados entre as extremidades desejadas.

A Figura 7.21 indica a estrutura lógica utilizada para dar suporte às trocas por InfiniBand. Para considerar o fato de que alguns dispositivos podem enviar dados mais rapidamente do que outros dispositivos de destino podem recebê-los, um par de filas nas duas extremidades de cada enlace mantém os dados que entram e saem temporariamente em buffers. As filas podem estar localizadas no adaptador do canal ou na memória do dispositivo conectado. Um par de filas separado é usado para cada pista virtual. O sistema principal utiliza essas filas da

Figura 7.21 Pilha de protocolos de comunicação InfiniBand



IB = InfiniBand
WQE = work queue element (elemento de fila de trabalho)
CQE = completion queue entry (entrada de fila de término)
PF = par de filas

seguinte forma. Ele coloca uma transação, chamada elemento de fila de trabalho (WQE — *work queue element*) na fila de emissão ou recepção do par de filas. As duas WQEs mais importantes são SEND e RECEIVE. Para uma operação SEND, a WQE especifica um bloco de dados no espaço de memória do dispositivo para o hardware enviar ao destino. Uma WQE RECEIVE especifica onde o hardware deve colocar dados recebidos de outro dispositivo quando esse consumidor executa uma operação SEND. O adaptador de canal processa cada WQE postada em ordem de prioridade adequada e gera uma entrada de fila de término (CQE — *completion queue entry*) para indicar o estado de término.

A Figura 7.21 também indica que uma arquitetura de protocolo é utilizada. Ela consiste em quatro camadas:

- **Física:** a especificação da camada física define três velocidades de conexão (1X, 4X e 12X), dando taxas de transmissão de 2,5, 10 e 30 Gbps, respectivamente (Tabela 7.3). A camada física também define o meio físico, incluindo cobre e fibra óptica.
- **Enlace:** essa camada define a estrutura básica do pacote, usada para trocar dados, incluindo um esquema de endereçamento que atribui um endereço exclusivo de conexão a cada dispositivo em uma sub-rede. Esse nível inclui a lógica para configurar pistas virtuais e trocar dados pelos comutadores da origem ao destino dentro de uma sub-rede. A estrutura do pacote inclui um código de detecção de erro para oferecer confiabilidade.
- **Rede:** a camada de rede direciona os pacotes entre diferentes sub-redes InfiniBand.
- **Transporte:** a camada de transporte oferece mecanismo de confiabilidade para transferências de pacotes de ponta a ponta entre uma ou mais sub-redes.



7.8 Leitura recomendada e sites Web

Uma boa discussão sobre os módulos de E/S da Intel e sua arquitetura, incluindo 82C59A, 82C55A, e 8237A, pode ser encontrada em Brey (2009^b) e Mazidi e Mazidi (2003^c).

FireWire é abordado com bastante detalhe em Anderson (1998^d). Wickelgren (1997^e) e Thompson (2000^f) oferecem visões gerais do FireWire.

InfiniBand é abordado com bastante detalhe em Shanley (2003^g) e Futral (2001^h). Kagan (2001ⁱ) oferece uma visão geral concisa.



Sites Web recomendados

T10 Home Page: T10 é um comitê técnico do *National Committee on Information Technology Standards*, e é responsável pelas interfaces de nível mais baixo. Seu principal trabalho é a *Small Computer System Interface* (SCSI).

1394 Trade Association: inclui informações técnicas e indicadores de fornecedores de FireWire.

Infiniband Trade Association: inclui informações técnicas e indicadores de fornecedores de Infiniband.

National Facility for I/O Characterization and Optimization: uma instalação dedicada a educação e pesquisa na área de projeto e desempenho de E/S. Ferramentas e tutoriais úteis.

Tabela 7.3 Enlaces InfiniBand e taxas de vazão de dados

Enlace	Taxa de sinal (unidirecional)	Capacidade usável (80% da taxa de sinal)	Vazão de dados efetiva (envio + recebimento)
largura 1	2,5 Gbps	2 Gbps (250 Mbps)	(250 + 250) Mbps
larguras 4	10 Gbps	8 Gbps (1 Gbps)	(1 + 1) Gbps
larguras 12	30 Gbps	24 Gbps (3 Gbps)	(3 + 3) Gbps

Principais termos, perguntas de revisão e problemas

Principais termos

Roubo de ciclo	Canal de E/S	Canal multiplexador
Acesso direto à memória (DMA)	Comando de E/S	E/S paralela
FireWire	Módulo de E/S	Dispositivo periférico
InfiniBand	Processador de E/S	E/S programada
Interrupção	E/S independente	Canal seletor
E/S controlada por interrupção	E/S mapeada na memória	E/S serial

Perguntas de revisão

- 7.1 Liste três classificações gerais de dispositivos externos ou periféricos.
- 7.2 O que é o *International Reference Alphabet*?
- 7.3 Quais são as principais funções de um módulo de E/S?
- 7.4 Liste e defina resumidamente três técnicas para realizar E/S.
- 7.5 Qual é a diferença entre E/S mapeada na memória e E/S independente?
- 7.6 Quando ocorre uma interrupção de dispositivo, como o processador determina qual dispositivo emitiu a interrupção?
- 7.7 Quando um módulo de DMA toma o controle de um barramento, e enquanto ele retém o controle do barramento, o que o processador faz?

Problemas

- 7.1 Em um microprocessador típico, um endereço de E/S distinto é usado para se referir aos registradores de dados de E/S e um endereço distinto para os registradores de controle e estado em um controlador de E/S para determinado dispositivo. Esses registradores são conhecidos como **portas**. No Intel 8088, dois formatos de instrução de E/S são utilizados. Em um formato, o *opcode* de 8 bits especifica uma operação de E/S; isso é seguido por um endereço de porta de 8 bits. Outros *opcodes* de E/S implicam que o endereço de porta está no registrador DX de 16 bits. Quantas portas o 8088 pode endereçar em cada modo de endereçamento de E/S?
- 7.2 Um formato de instrução semelhante é usado na família de microprocessadores Zilog Z8000. Nesse caso, existe uma capacidade de endereçamento direto de porta, em que um endereço de porta de 16 bits faz parte da instrução, e uma capacidade de endereçamento indireto de porta, em que a instrução referencia um dos registradores de uso geral de 16 bits, que contém o endereço da porta. Quantas portas o Z8000 pode endereçar em cada modo de endereçamento de E/S?
- 7.3 O Z8000 também inclui uma capacidade de transferência de E/S em bloco que, diferente do DMA, está sob o controle direto do processador. As instruções de transferência em bloco especificam um registrador de endereço de porta (Rp), um registrador de contagem (Rc) e um registrador de destino (Rd). Rd contém o endereço da memória principal em que o primeiro byte lido da porta de entrada deve ser armazenado. Rc é qualquer um dos registradores de uso geral de 16 bits. Que tamanho de bloco de dados pode ser transferido?
- 7.4 Considere um microprocessador que tenha uma instrução de transferência de E/S em bloco, como aquela encontrada no Z8000. Após sua primeira execução, essa instrução leva cinco ciclos de clock para ser reexecutada. Porém, se empregarmos uma instrução de E/S sem bloqueio, isso exigirá um total de 20 ciclos de clock para a busca e execução. Calcule o aumento na velocidade com a instrução de E/S em bloco para transferir blocos de 128 bytes.
- 7.5 Um sistema é baseado em um microprocessador de 8 bits e tem dois dispositivos de E/S. Os controladores de E/S para esse sistema utilizam registradores separados para controle e estado. Os dois dispositivos tratam dos dados com 1 byte de cada vez. O primeiro dispositivo tem duas linhas de estado e três linhas de controle. O segundo dispositivo tem três linhas de estado e quatro linhas de controle.
 - a. Quantos registradores do módulo de controle de E/S de 8 bits precisamos para leitura de estado e controle de cada dispositivo?
 - b. Qual é o número total de registradores de módulo de controle necessários, dado que o primeiro dispositivo está apenas no dispositivo de saída?

- c. Quantos endereços distintos são necessários para controlar os dois dispositivos?
- 7.6** Para a E/S programada, a Figura 7.5 indica que o processador fica preso em um loop de espera verificando o estado de um dispositivo de E/S. Para aumentar a eficiência, o software de E/S poderia ser escrito de modo que o processador periodicamente verificasse o estado do dispositivo. Se o dispositivo não estiver pronto, o processador poderá executar outras tarefas. Após algum intervalo, o processador volta a verificar o estado novamente.
- Considere o esquema acima para a saída de dados um caractere de cada vez para uma impressora que opera a 10 caracteres por segundo (cps). O que acontecerá se seu estado foi verificado a cada 200 ms?
 - Em seguida, considere um teclado com um buffer de caracteres. Na média, os caracteres são inseridos a uma taxa de 10 cps. Porém, o intervalo de tempo entre dois toques de tecla consecutivos pode ser tão curto quanto 60 ms. Em que frequência o teclado deve ser verificado pelo programa de E/S?
- 7.7** Um microprocessador verificar o estado de um dispositivo de saída a cada 20 ms. Isso é feito por meio de um timer alertando o processador a cada 20 ms. A interface do dispositivo inclui duas portas: uma para estado e uma para saída de dados. Quanto tempo é necessário para verificar e atender ao dispositivo dada uma taxa de clock de 8 MHz? Suponha, para simplificar, que todos os ciclos de instrução pertinentes sejam de 12 ciclos de clock.
- 7.8** Na Seção 7.3, listamos uma vantagem e uma desvantagem da E/S mapeada na memória, comparada com a E/S independente. Liste mais duas vantagens e mais duas desvantagens.
- 7.9** Um sistema em particular é controlado por um operador por meio de comandos digitados em um teclado. O número médio de comandos entrados em um intervalo de 8 horas é 60.
- Suponha que o processador verifique o teclado a cada 100 ms. Quantas vezes o teclado será verificado em um período de 8 horas?
 - Por que fração o número de verificações do processador ao teclado seria reduzido se fosse usada a E/S controlada por interrupção?
- 7.10** Considere um sistema empregando a E/S controlada por interrupção para determinado dispositivo que transfere dados em uma média de 8 KB/s de forma contínua.
- Suponha que o processamento da interrupção gaste 100 ms (ou seja, o tempo para saltar até a rotina de tratamento de interrupção (ISR), executá-la e retornar ao programa principal). Determine que fração do tempo do processador é consumida por esse dispositivo de E/S se ele interromper a cada byte.
 - Agora, suponha que o dispositivo tenha dois buffers de 16 bytes e interrompa o processador quando um dos buffers estiver cheio. Naturalmente, o processamento da interrupção leva mais tempo, pois a ISR precisa transferir 16 bytes. Ao executar a ISR, o processador leva cerca de 8 ms para a transferência de cada byte. Determine que fração do tempo do processador é consumida por esse dispositivo de E/S nesse caso.
 - Agora, suponha que o processador seja equipado com uma instrução de E/S para transferência em bloco, como aquela encontrada no Z8000. Isso permite que a ISR associada transfira cada byte de um bloco em apenas 2 ms. Determine que fração do tempo do processador é consumida por esse dispositivo de E/S nesse caso.
- 7.11** Em praticamente todos os sistemas que incluem módulos de DMA, o acesso por DMA à memória principal recebe prioridade mais alta que o acesso da CPU à memória principal. Por quê?
- 7.12** Um módulo de DMA está transferindo caracteres para a memória usando o roubo de ciclo, a partir de um dispositivo transmitindo a 9 600 bps. O processador está buscando instruções na taxa de 1 milhão de instruções por segundo (1 MIPS). Por quanto tempo o processador será atrasado devido à atividade de DMA?
- 7.13** Considere um sistema em que os ciclos do barramento levem 500 ns. A transferência do controle do barramento em qualquer direção, do processador para o dispositivo de E/S ou vice-versa, leva 250 ns. Um dos dispositivos de E/S tem uma taxa de transferência de 50 KB/s e emprega DMA. Os dados são transferidos um byte de cada vez.
- Suponha que empreguemos DMA em um modo de bloco. Ou seja, a interface de DMA ganha controle do barramento antes do início de uma transferência em bloco e mantém o controle do barramento até que o bloco inteiro seja transferido. Por quanto tempo o dispositivo prenderia o barramento ao transferir um bloco de 128 bytes?
 - Repita o cálculo para o modo de roubo de ciclo.
- 7.14** O exame do diagrama de tempo do 8237A indica que, quando uma transferência em bloco é iniciada, ela exige três ciclos de clock do barramento por ciclo de DMA. Durante o ciclo de DMA, o 8237A transfere um byte de informações entre a memória e o dispositivo de E/S.
- Suponha que usemos uma taxa de clock de 6 MHz no 8237A. Quanto tempo é necessário para transferir um byte?
 - Qual seria a taxa de transferência de dados máxima alcançável?

- c. Suponha que a memória não seja rápida o suficiente e que tenhamos que inserir dois estados de espera por ciclo de DMA. Qual será a taxa de transferência de dados real?

7.15 Suponha que, no sistema do problema anterior, um ciclo de memória leve 750 ns. Para que valor poderíamos reduzir a taxa de clock do barramento sem afetar a taxa de transferência de dados alcançável?

7.16 Um controlador de DMA atende a quatro enlaces de telecomunicação apenas de recepção (um por canal de DMA) tendo uma velocidade de 64 Kbps cada.

- Você operaria o controlador no modo bloco ou no modo de roubo de ciclo?
- Que esquema de prioridade você empregaria para o atendimento dos canais de DMA?

7.17 Um computador de 32 bits tem dois canais seletores e um canal multiplexador. Cada canal seletor aceita duas unidades de disco magnético e duas unidades de fita magnética. O canal multiplexador tem duas impressoras de linha, duas leitoras de cartão e 10 terminais VDT conectados a ele. Considere as seguintes taxas de transferência:

Unidade de disco	800 KBytes/s
Unidade de fita magnética	200 KBytes/s
Impressora de linha	6,6 KBytes/s
Leitora de cartões	1,2 KBytes/s
VDT	1 KBytes/s

Estime a taxa de transferência de E/S agregada máxima nesse sistema.

7.18 Um computador consiste em um processador de E/S e um dispositivo D conectado à memória principal M por meio de um barramento compartilhado com uma largura de barramento de dados de uma palavra. O processador pode executar um máximo de 10^6 instruções por segundo. Uma instrução média requer cinco ciclos de máquina, três dos quais utilizam o barramento da memória. Uma operação de leitura ou escrita da memória utiliza um ciclo de máquina. Suponha que o processador esteja continuamente executando programas em “segundo plano” que exigem 95% de sua taxa de execução de instrução, mas não quaisquer instruções de E/S. Suponha que um ciclo de processador seja igual a um ciclo de barramento. Agora, suponha que o dispositivo de E/S deva ser usado para transferir grandes blocos de dados entre M e D.

- Se a E/S programada for usada e cada transferência de E/S de uma palavra exigir que o processador execute duas instruções, estime a taxa de transferência de dados de E/S máxima, em palavras por segundo, possível através de D.
- Estime a mesma taxa se o DMA for utilizado.

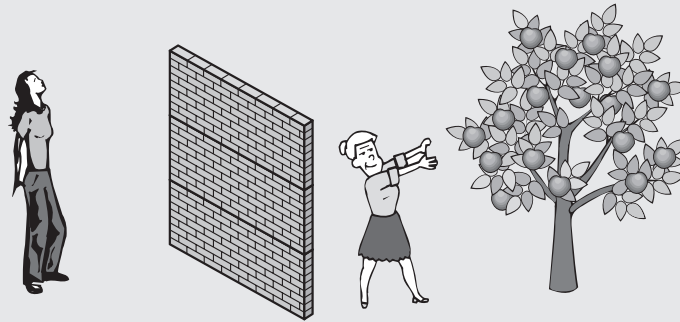
7.19 Uma fonte de dados produz caracteres IRA de 7 bits e, a cada um deles, é anexado um bit de paridade. Derive uma expressão para a taxa de dados efetivos máxima (taxa de bits de dados IRA) por uma linha de R bps para os seguintes:

- Transmissão assíncrona, com um stop bit de 1,5 unidades.
- Transmissão síncrona de bit, com um *frame* consistindo em 48 bits de controle e 128 bits de informação.
- O mesmo que (b), com um campo de informação de 1024 bits.
- Síncrono de caractere, com 9 caracteres de controle por *frame* e 16 caracteres de informação.
- O mesmo que (d), com 128 caracteres de informação.

7.20 O problema a seguir é baseado em uma ilustração sugerida dos mecanismos de E/S em Eckert (1990¹) (Figura 7.22):

Duas mulheres estão em cada lado de uma cerca alta. Uma das mulheres, chamada Servidora de maçã, tem uma bela macieira carregada de deliciosas maçãs, crescendo em seu lado da cerca; ela está contente por fornecer maçãs à outra mulher sempre que preciso. A outra mulher, chamada Comedora de maçã, gosta de comer maçãs, mas não tem nenhuma. Na verdade, ela precisa comer suas maçãs em uma velocidade fixa (uma maçã por dia é o suficiente para afastar doenças). Se ela as comer mais rápido do que essa velocidade, ficará doente. Se comer mais lentamente, terá desnutrição. Nenhuma das duas mulheres pode falar e, portanto, o problema é levar as maçãs da Servidora de maçã para a Comedora de maçã na velocidade correta.

- Suponha que haja um relógio despertador em cima da cerca, e que o relógio pode ter várias configurações de alarme. Como o relógio pode ser usado para solucionar o problema? Desenhe um diagrama de temporização para ilustrar a solução.
- Agora, suponha que não haja um relógio despertador. Em vez disso, a Comedora de maçã tem uma bandeira que ela pode acenar sempre que precisar de uma maçã. Sugira uma nova solução. Seria útil que a Servidora de maçã também tivesse uma bandeira? Se for, incorpore isso à solução. Discuta as desvantagens dessa técnica.
- Agora, retire a bandeira e considere a existência de uma longa corda. Sugira uma solução que seja superior à de (b) usando a corda.

Figura 7.22 O problema da maçã

7.21 Suponha que um microprocessador de 16 bits e dois de 8 bits devam ser ligados a um barramento do sistema. Os seguintes detalhes são dados:

1. Todos os microprocessadores têm os recursos de hardware necessários para qualquer tipo de transferência de dados: E/S programada, E/S controlada por interrupção e DMA.
2. Todos os microprocessadores têm um barramento de endereço de 16 bits.
3. Duas placas de memória, cada uma com 64 KBytes de capacidade, são interligadas ao barramento. O projetista deseja usar uma memória compartilhada que seja a maior possível.
4. O barramento do sistema admite um máximo de quatro linhas de interrupção e uma linha de DMA. Faça quaisquer outras suposições necessárias e:
 - a. Dê as especificações do barramento do sistema em termos do número e tipos de linhas.
 - b. Descreva um protocolo possível para a comunicação no barramento (ou seja, leitura-escrita, interrupção e sequências de DMA).
 - c. Explique como os dispositivos mencionados são interligados ao barramento do sistema.

Referências

- a STALLINGS, W. *Operating systems, internals and design principles, 6th Edition*. Upper Saddle River, NJ: Prentice Hall, 2009.
- b BREY, B. *The Intel microprocessors: 8086/8066, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro Processor, Pentium II, Pentium III, Pentium 4 and Core2 with 64-bit extensions*. Upper Saddle River, NJ: Prentice Hall, 2009.
- c MAZIDI, M. e MAZIDI, J. *The 80x86 IBM PC and compatible computers: assembly language, design and interfacing*. Upper Saddle River, NJ: Prentice Hall, 2003.
- d ANDERSON, D. *FireWire system architecture*. Reading, MA: Addison-Wesley, 1998.
- e WICKELGREN, I. "The facts about Fire Wire". *IEEE Spectrum*, abr. 1997.
- f THOMPSON, D. "IEEE 1394: changing the way we do multimedia communications". *IEEE Multimedia*, abr./jun. 2000.
- g SHANLEY, T. *InfiniBand network architecture*. Reading, MA: Addison-Wesley, 2003.
- h FUTRAL, W. *InfiniBand architecture: development and deployment*. Hillsboro, OR: Intel Press, 2001.
- i KAGAN, M. "InfiniBand: thinking outside the box design". *Communications System Design*, set. 2001. Disponível em: <www.csdmag.com>.
- j ECKERT, R. "Communication between computers and peripheral devices — An analogy". *ACM SIGCSE Bulletin*, set. 1990.